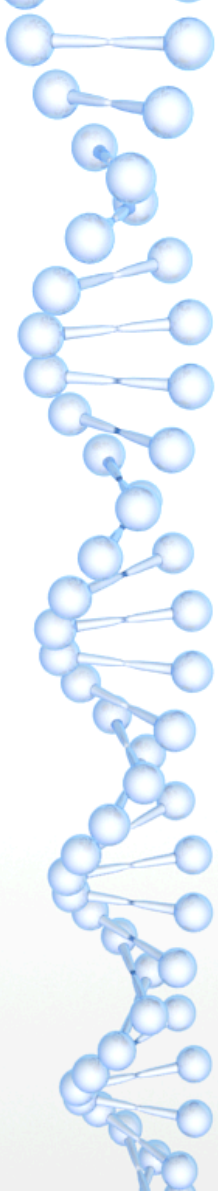
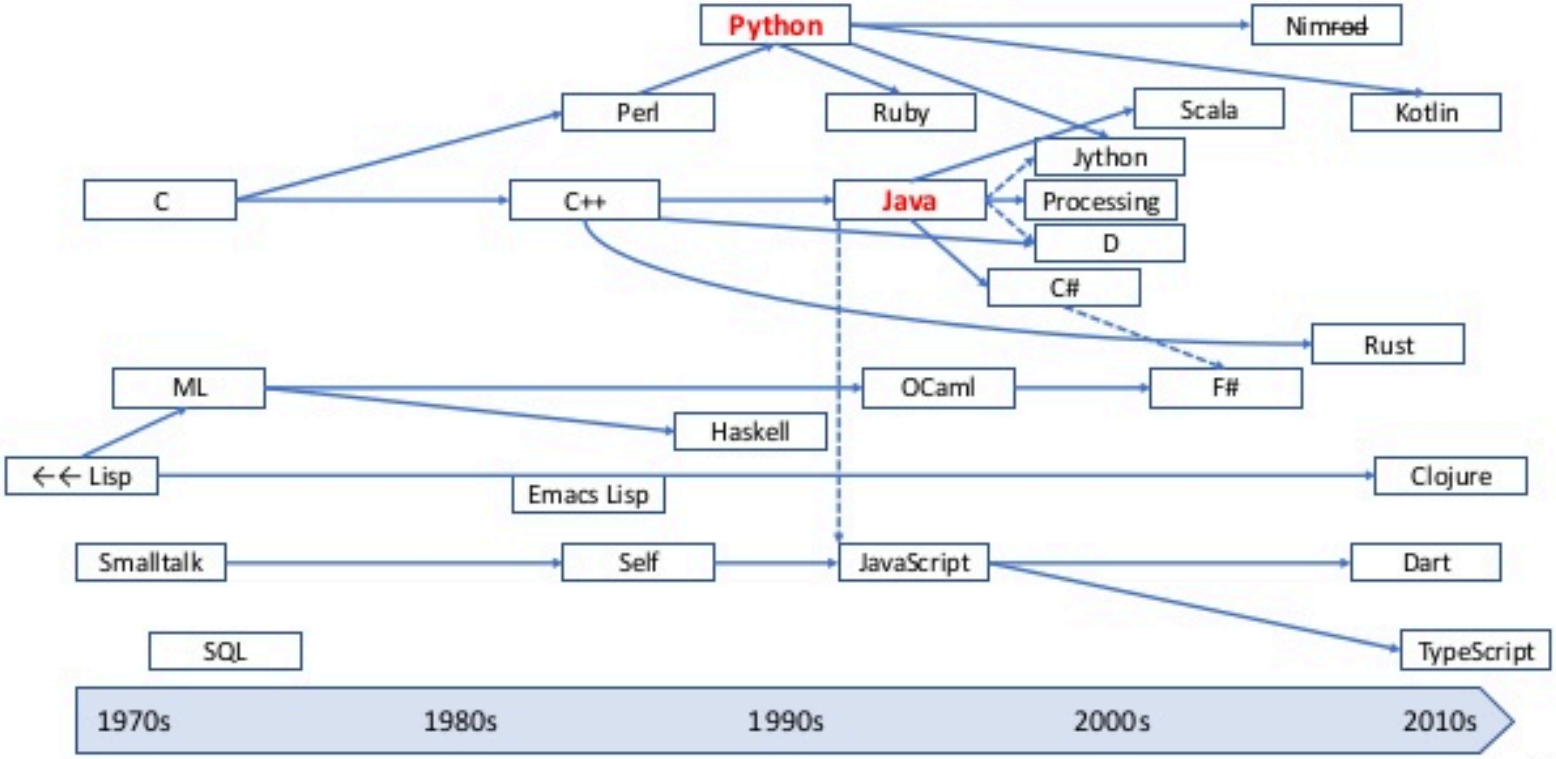


Introduction to Python

Ivan Lanese

A Brief Family Tree of Some Programming Languages



License: CC BY-SA 4.0 (except images), by Jay Coskey

Programming languages

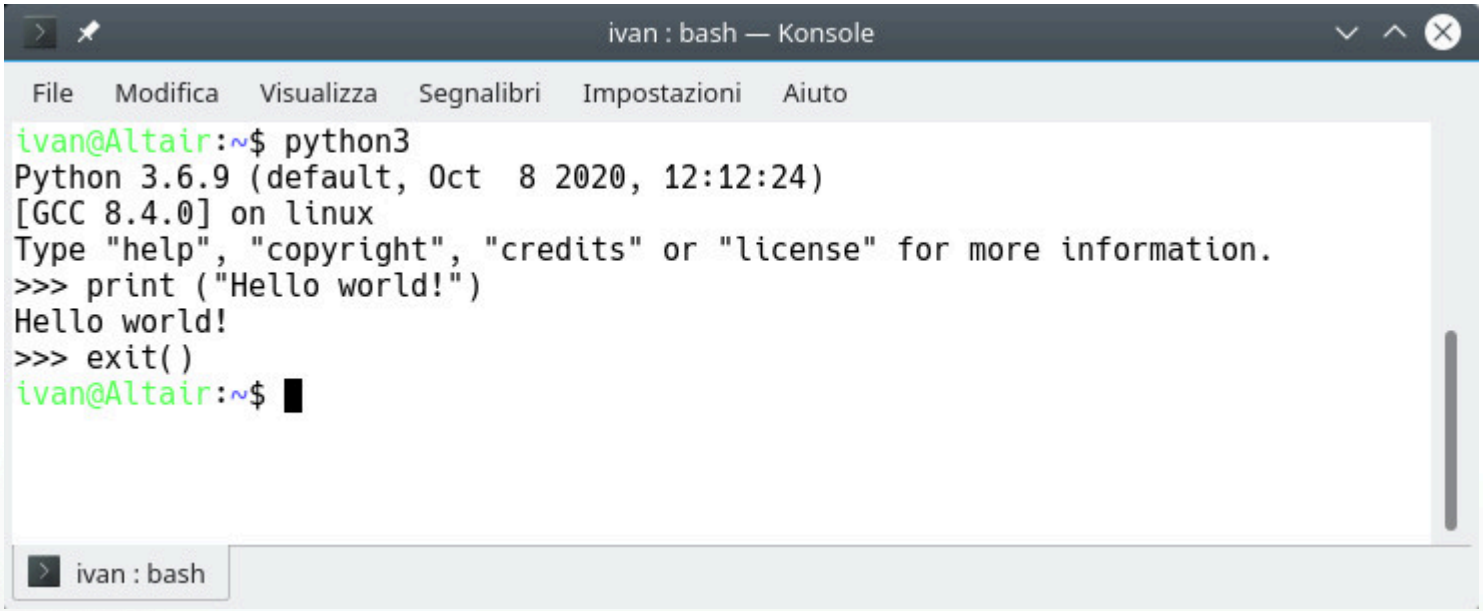
- A few thousands of programming languages exist
- You may have heard of some of them:
  - FORTRAN, BASIC, COBOL (old)
  - C, C++, C#, JAVA, JAVASCRIPT, PYTHON (mainstream)
  - RUST, ERLANG, GO, HASKELL (peculiar approaches)
- Python is particularly well suited for non-experts and rapid prototyping
- Python is used in many areas, particularly machine learning and AI, data analysis, and web development

Programming basic terminology

- Code or source code: the sequence of instructions composing a program (saved in files .py)
- Syntax: the set of legal structures and commands that can be used in a particular (programming) language
- Output: the messages printed to the user by a program
- Console: the text box onto which output is printed
  - Console may just be the shell, but some IDEs pop up the console as an external window, and others contain their own console

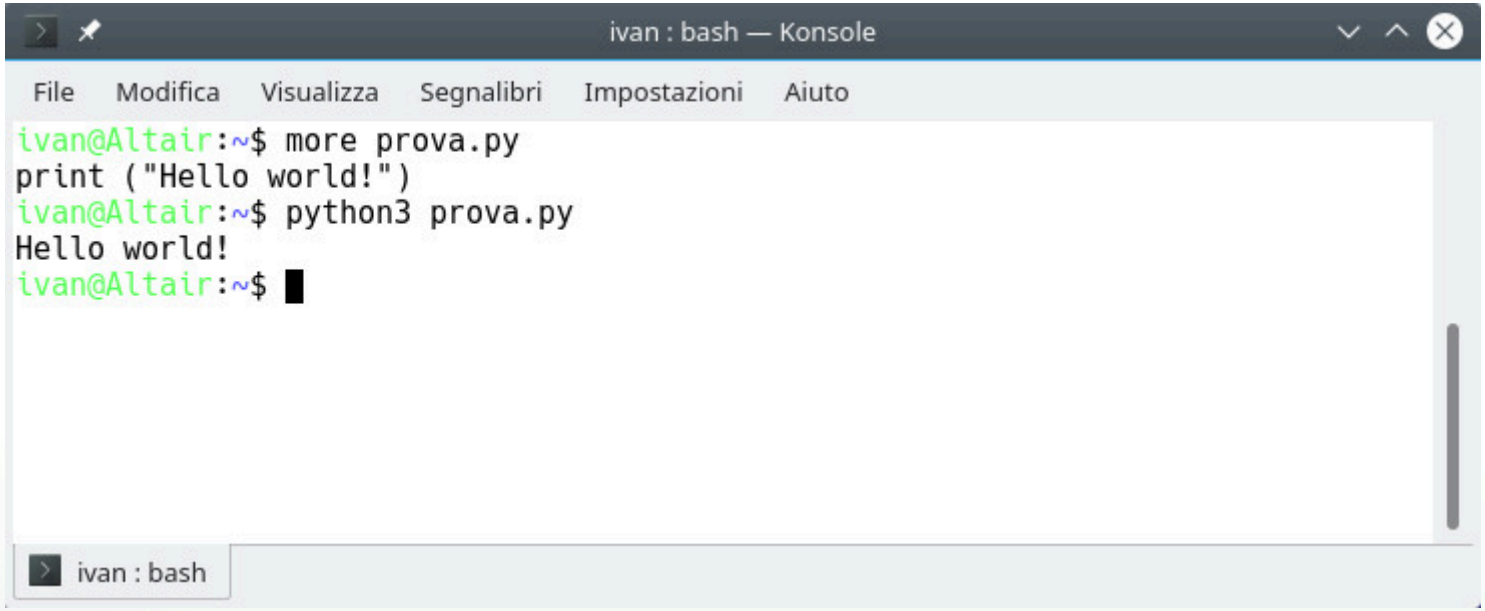
Running our first program

- We start by writing it inside the python interpreter



Running our first program, again

- Now, we write it in a .py file and we run it



Interpreters and compilers

- Interpreters like python3 read a program source code and execute it directly
  - You cannot run a python program without the interpreter
- Other languages (such as c) use compilers: a compiler translates source code into machine code
  - Machine code can be executed by the machine directly
- There are also compilers for python, but they are less used
  - We will not use them during the course

Let's start: expressions

- We can use python to do mathematical calculations
- **Expression**: a value or a set of operations to compute a value
- `5` or `4 + 3` or `(3.9 * 4)/(3 - 6)`
- Arithmetic operators:

  `+` `-` `*` addition, subtraction/negation, multiplication

  `/` `//` floating-point division, integer division

  `%` integer division remainder

  `**` exponentiation

Precedence

- Operations are computed in a pre-defined order
- `**` has precedence over `*`, `/`, `//`, which have precedence over `+` and `-`
- If operations have the same precedence then evaluation is from left to right
- `1 + 3 * 4` is `13`
  `10 / 5 * 2` is `4.0`
- Parenthesis can be added to force evaluation
- `(1+3) * 4` is `16`

Integers and floating points

- Python can use both integers (`5`, `-10`) and floating points (`3.4`, `4e-5`)
- Floating points approximate reals
- Operators and precedence rules are the same in both cases
- When integers and floating points are mixed, the result is a floating point
  `3 + 2.3` is `5.3`
- The result of `/` is always a floating point

Integer division

- When we divide numbers with // , we only get the integer part of the quotient.
- `35 // 5` is `7`
- `84 // 10` is `8`
- `156.2 // 100` is `1.0`

- The % operator computes the remainder from a division of integers.
- `35 % 5` is `0`
- `84 % 10` is `4`
- `156.2 % 100` is `56.199999999999999` (approximation error)

Math functions and constants

Function	Description
`abs(`***value***`)`	absolute value
`ceil(`***value***`)`	rounds up
`cos(`***value***`)`	cosine, in radians
`floor(`***value***`)`	rounds down
`log(`***value***`)`	logarithm, base *e*
`log10(`***value***`)`	logarithm, base 10
`max(`***value1***`, `***value2***`)`	largest of two values
`min(`***value1***`, `***value2***`)`	smallest of two values
`round(`***value***`)`	nearest whole number
`sin(`***value***`)`	sine, in radians
`sqrt(`***value***`)`	square root

Constant	Value
e	2.7182818...
pi	3.1415926...

To use many of them, you must write at the top of your program:

```
from math import *
```

Variables

- **Variable**: A named piece of memory that can store a value.
- Usage: Compute an expression's result, store that result into a variable, and use that variable later in the program.
- **Assignment statement**: Stores a value into a variable.
- Syntax: name = expression
- Examples: `x = 5`
  `gpa = 3.14`

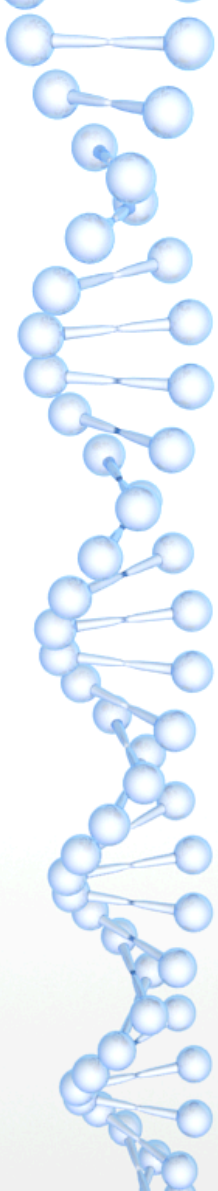
- A variable that has been given a value can be used in expressions.
- `x + 4` is `9`

Printing results

- If you execute a program interactively (that is, by writing it inside the interpreter), python prints the result of each line
- Otherwise, you need to add print statements to display the results
- print ("Message")
- print (Expression)
- Prints the given text message or expression value on the console, and moves the cursor down to the next line.
- print (Item1, Item2, ..., ItemN)
- Prints several messages and/or expressions on the same line.

Exercise

- Evaluate the quadratic expression ax^2+bx+c for given a, b, c and x, and print the result on the screen.



Strings in Python

Ivan Lanese

Computers store text as strings

- A string is a sequence of characters
- `>>> S = "GATTACA"`

	0	1	2	3	4	5	6
S →	G	A	T	T	A	C	A

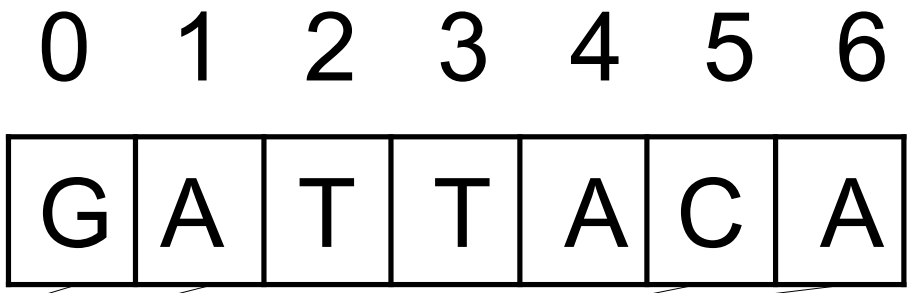
Why are strings important?

- DNA sequences are strings
  - `..catgaaggaa ccacagccca gagcaccaag..`
- Database records frequently contain strings
  - `LOCUS AC005138`
  - `DEFINITION Homo sapiens chromosome 17, clone hRPK.261_A_13, complete sequence`
  - `AUTHORS Birren,B., Fasman,K., Linton,L., Nusbaum,C. and Lander,E.`
- An HTML file is a (large) string

Extracting characters

- You can extract characters from strings by position

```
>>> s = "GATTACA"
>>> s[0]
'G'
>>> s[1]
'A'
>>> s[-1]
'A'
>>> s[-2]
'C'
>>> s[7]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range
>>>
```



Extracting substrings

- You can extract substrings by giving initial and final positions and step
- `s[start:end:step]`

0	1	2	3	4	5	6
G	A	T	T	A	C	A

```
>>> s[1:3]
'AT'
>>> s[:3]
'GAT'
>>> s[4:]
'ACA'
>>> s[3:6]
'TAC'
>>> s[:]
'GATTACA'
```

```
>>> s[::2]
'GTAA'
>>>s[-2:2:-1]
'CAT'
>>>
```

Creating strings

- Strings start and end with a single (‘) or double quote (“) characters (they must be the same)

```
"This is a string"
"This is another string"
""
"Strings can be in double quotes"
'Or in single quotes.'
"There's no difference."
'Okay, there\'s a small one.'
```

Special characters and escape sequences

- Backslashes (\) are used to introduce special characters

```
>>> s = 'Okay, there\'s a small one.'
```

- The backslash “escapes” the following ‘

```
>>> print(s)
Okay, there's a small one.
```

Some special characters

Escape sequence	Meaning
\\	Backslash (keeps a \)
\’	Single quote (keeps the ‘)
\”	Double quote (keeps the ”)
\n	Newline
\t	Tab

Working with strings

```
>>> len("GATTACA")
7
>>> "GAT" + "TACA"
'GATTACA'
>>> "A" * 10
'AAAAAAAAAA'
>>> "G" in "GATTACA"
True
>>> "GAT" in "GATTACA"
True
>>> "AGT" in "GATTACA"
False
>>> "GATTACA".find("ATT")
1
>>> "GATTACA".count("T")
2
>>>
```

length

concatenation

repetition

substring test

Object-oriented syntax, we will discuss it in detail towards the end of the course

substring location

(disjoint) substring count

Type conversion (a.k.a. type cast)

```
>>> "38" + 5
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
  TypeError: must be str, not int
>>> int("38") + 5
43
>>> "38" + str(5)
'385'
>>> int("38"), str(5)
(38, '5')
>>> int("2.71828")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: '2.71828'
>>> float("2.71828")
2.71828
>>> int(2.71828)
2
>>>
```

Changing a string? No, thanks!

- Characters inside strings cannot be modified

```
>>> s = "GATTACA"
>>> s[3] = "C"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
```

- A string variable can be re-assigned

```
>>> s = s[:3] + "C" + s[4:]
>>> s
'GATCACA'
>>> s = "PIPPO"
>>> s
'PIPPO'
```

Taking a string from the user

- Strings can be taken from the user using function `input("prompt")`

```
>>> name=input("Insert your name:”)
Insert your name:Ivan
>>> print(name)
Ivan
```

User input

- Variable `name` will not contain the final newline

Working with strings (2nd part)

```
>>> "GATTACA".lower()
'gattaca'
>>> "gattaca".upper()
'GATTACA'
>>> "GATTACA".replace("G", "U")
'UATTACA'
>>> "GATTACA".replace("T", "U")
'GAUUACA'
>>> "GATTACA".replace("AT", "*")
'G*TACA'
>>> "GATTACA".startswith("GA")
True
>>> "GATTACA".startswith("ga")
False
>>>
```

Exercise

- Write a program that takes a string for a DNA sequence and analyses it as follows
- On input `ATTAC` it produces the output below:

```
Sequence is: ATTAC
It is 5 bases long
adenine: 2
thymine: 2
cytosine: 1
guanine: 0
```

Exercise

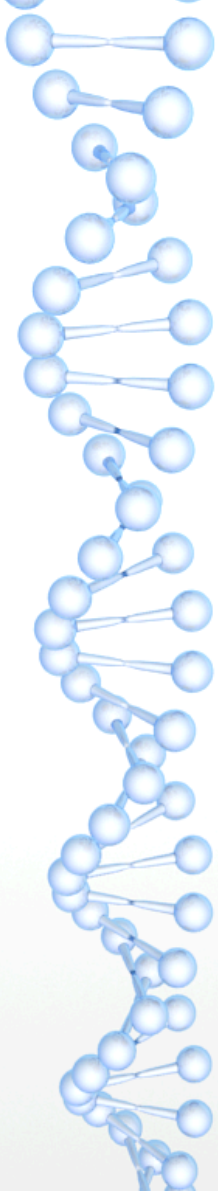
- Modify the previous program to accept both uppercase and lowercase letters.
- On input `ATtaC` it produces the output below:

```
Sequence is: ATtaC
It is 5 bases long
adenine: 2
thymine: 2
cytosine: 1
guanine: 0
```

Exercise

- Modify the previous program to also print the number of unknown characters.
- On input `ATUt*aC` it produces the output below:

```
Sequence is: ATUt*aC
It is 7 bases long
adenine: 2
thymine: 2
cytosine: 1
guanine: 0
unknown: 2
```



Lists

Ivan Lanese

Lists

- Lists are ordered collections of data
- creating an empty list

```
>>> data = []
>>> data
[]
>>> data.append("Hello")
>>> print(data)
['Hello']
>>> data.append(5)
>>> print(data)
['Hello',5]
>>> data.append([9,8,7])
>>> print(data)
['Hello',5,[9,8,7]]
>>> data.extend([4,5,6])
>>> print(data)
['Hello',5,[9,8,7],4,5,6]
>>>
```

`append` adds an element at the end

elements can even be lists

`extend` appends each element of a list

Lists and strings are similar

- Strings

```
>>> s = "ATCG"
>>> print(s[0])
A
>>> print(s[-1])
G
>>> print(s[2:])
CG
>>> print("C" in s)
True
>>> s * 3
'ATCGATCGATCG'
>>> s[9]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range
>>>
```

- Lists

```
>>> L = ["adenine", "thymine", "cytosine",
"guanine"]
>>> print(L[0])
adenine
>>> print(L[-1])
guanine
>>> print(L[2:])
['cytosine', 'guanine']
>>> print("cytosine" in L)
True
>>> L * 3
['adenine', 'thymine', 'cytosine',
'guanine',  'adenine', 'thymine',
'cytosine', 'guanine',  'adenine',
'thymine', 'cytosine', 'guanine']
>>> L[9]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: list index out of range
>>>
```

Lists can be updated, strings cannot

- Strings

```
>>> s = "ATCG"
>>> print(s)
ATCG
>>> s[1] = "U"
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not
support item assignment
>>> s.reverse()
Traceback (most recent call last):
   File "<stdin>", line 1, in <module>
AttributeError: 'str' object has no
attribute  'reverse'
>>> print(s[::-1])
GCTA
>>> print(s)
ATCG
>>>
```

- Lists

```
>>> L = ["adenine", "thymine",
"cytosine",  "guanine"]
>>> print(L)
['adenine', 'thymine', 'cytosine',
'guanine']
>>> L[1] = "uracil"
>>> print(L)
['adenine', 'uracil', 'cytosine',
'guanine']
>>> L.reverse()
>>> print(L)
['guanine', 'cytosine', 'uracil',
'adenine']
>>> del(L[0])
>>> print(L)
['cytosine', 'uracil', 'adenine']
>>>
```

Immutability and method invocation

- When we invoke a method on a string, and the result is a string, it is returned as a value
- We need to assign it to a variable (the original string is unchanged)
-
```
>>> s = "HELLO"
>>> s1 = s.lower()
>>> print(s1,s)
hello HELLO
```
- When we invoke a method on a list, and the result is a list, it is normally modified in place
- No need to assign the result to a variable, and if we do then the variable gets the value `None` (for no value)
-
```
>>> l = [1,2,3,4]
>>> l1=l.reverse()
>>> print(l1)
None [4, 3, 2, 1]
```

Lists can hold any kind of data

```
>>> L = ["", 1, "two", 3.0, ["fem", [6j+2], []]]
>>> len(L)
5
>>> print(L[-1])
['fem', [2+6j], []]
>>> len(L[-1])
3
>>> print L[-1][-1]
[]
>>> len(L[-1][-1])
0
>>>
```

Working with lists

- ```
  >>> L = ["thymine", "cytosine", "guanine"]
  >>> L.insert(0, "adenine")
  >>> print(L)
  ['adenine', 'thymine', 'cytosine', 'guanine']
  >>> L.insert(2, "uracil")
  >>> print(L)
  ['adenine', 'thymine', 'uracil', 'cytosine', 'guanine']
  >>> print(L[:2])
  ['adenine', 'thymine']
  >>> L[:2] = ["A", "T"]
  >>> print(L)
  ['A', 'T', 'uracil', 'cytosine', 'guanine']
  >>> L[:2] = []
  >>> print(L)
  ['uracil', 'cytosine', 'guanine']
  >>> L[:] = ["A", "T", "C", "G"]
  >>> print(L)
  ['A', 'T', 'C', 'G']
  >>>
  ```

insert in selected position

replace selected sublist
  ```

Exercise

- Write a program that takes in input a string composed by two words separated by a single space such as `"AAL532906 aaatatcccaattcagtatgcgctgagta"` and builds a list containing the two words.

An easier solution

- `split()` allows you to solve exactly this problem
- It is also more general: it works with any number of words
-
```
>>> fields = s.split()
>>> print(fields)
['AAL532906',
'aaaatagtcaaatatatcccaattcagtatgcgctgagta']
>>> print(fields[0])
AAL532906
>>> print(len(fields[1]))
40
>>>
```

More `split()` examples

- ```
  >>> protein = "ALA PRO ILU CYS"
  >>> residues = protein.split()
  >>> print(residues)
  ['ALA', 'PRO', 'ILU', 'CYS']
  >>>
  >>> protein = " ALA PRO   ILU     CYS     \n"
  >>> print protein.split()
  ['ALA', 'PRO', 'ILU', 'CYS']
  ```

`split()` splits on any form of whitespace

`split(s)` splits on occurrences of `s`

- ```
  >>> print "HIS-GLU-PHE-ASP".split("-")
  ['HIS', 'GLU', 'PHE', 'ASP']
  >>>
  ```

From lists to strings

- `join()` is the opposite of `split()`
-
```
>>> L1 = ["Asp", "Gly", "Gln", "Pro", "Val"]
>>> print("-".join(L1))
Asp-Gly-Gln-Pro-Val
>>> print("**".join(L1))
Asp**Gly**Gln**Pro**Val
>>> print("\n".join(L1))
Asp
Gly
Gln
Pro
Val
>>>
```

Separator first,
list second

The `for` loop

Ivan Lanese

Loops

- Programs are very good at doing the same computation on large collections of data
  - E.g., on each element of a given list
- This can be done using either loops or recursive functions
- We will now study the `for` loop, later on in the course we will also discuss `while` loops and recursive functions

The `for` loop

- ```
  >>> for name in ["Andrew", "Tsanwani", "Marco",
  "Sophie"]:
  ...     print("Hello,",name)
  ...
  Hello, Andrew
  Hello, Tsanwani
  Hello, Marco
  Hello, Sophie
  >>>
  ```
- Python3 does not answer after the first line
- The first line introduces a new block: you will get an answer only at the end of the block
- By a newline in the third line you tell python3 that the block is ended
  ```

The `for` loop

- ```
  >>> for name in ["Andrew", "Tsanwani", "Marco",
  "Sophie"]:
  ...     print("Hello,",name)
  ...
  Hello, Andrew
  Hello, Tsanwani
  Hello, Marco
  Hello, Sophie
  >>>
  ```

  Lines that introduce a new block end with :

- Python3 does not answer after the first line
- The first line introduces a new block: you will get an answer only at the end of the block
- By a newline in the third line you tell python3 that the block is ended
  ```

The `for` loop

- ```
  >>> for name in ["Andrew", "Tsanwani", "Marco",
  "Sophie"] :
  ...     print("Hello,",name)
  ...
  Hello, Andrew
  Hello, Tsanwani
  Hello, Marco
  Hello, Sophie
  >>>
  ```

Lines that introduce a new block end with :

All the lines of the new block (just one here) need to be indented (that is, preceded by some spaces) in the same way

- Python3 does not answer after the first line
- The first line introduces a new block: you will get an answer only at the end of the block
- By a newline in the third line you tell python3 that the block is ended
  ```

A block of two lines

```
>>> for name in ["Andrew", "Tsanwani", "Marco",
"Sophie"]:
...     print ("Hello,", name)
...     print ("Your name is", len(name), "letters long")
...
Hello, Andrew
Your name is 6 letters long
Hello, Tsanwani
Your name is 8 letters long
Hello, Marco
Your name is 5 letters long
Hello, Sophie
Your name is 6 letters long
>>>
```

Indentation errors

- Indentation is fundamental in python3
- Wrong indentation causes errors (or unexpected behaviours)
-
```
>>> a = 3
>>>   b = 5
  File "<stdin>", line 1
    b = 5
    ^
IndentationError: unexpected indent
>>> for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
...     print ("Hello,", name)
...         print("How are you?")
  File "<stdin>", line 3
    print("How are you?")
    ^
IndentationError: unexpected indent
>>> for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
...     print ("Hello,", name)
...   print("How are you?")
  File "<stdin>", line 3
    print("How are you?")
                         ^
IndentationError: unindent does not match any outer indentation level
```

Two similar programs

- What's the difference between the two programs below?
- ```
  for name in ["Andrew", "Tsanwani", "Marco",
  "Sophie"]:
      print("Hello,", name)
      print("How are you?")
  ```
- ```
  for name in ["Andrew", "Tsanwani", "Marco",
  "Sophie"]:
      print("Hello,", name)
  print("How are you?")
  ```

The `for` loop can iterate on strings as well

- ```
  seq = "ATGCAT"
  for letter in seq:
    print("Base:", letter)
  ```
- You get:
- ```
  Base: A
  Base: T
  Base: G
  Base: C
  Base: A
  Base: T
  ```

Numbering bases

- ```
  seq = "ATGCAT"
  n=0
  for letter in seq:
     print("Base", n, "is", letter)
     n = n + 1
  print ("The sequence has", n, "bases")
  ```
- You get:
- ```
  Base 0 is A
  Base 1 is T
  Base 2 is G
  Base 3 is C
  Base 4 is A
  Base 5 is T
  The sequence has 6 bases
  ```

The `range()` function

- The `range()` function allows to generate sequences of numbers, very good to iterate on using a `for` loop
-
```
>>> range(5)
range(0, 5)
>>> list(range(5))
[0, 1, 2, 3, 4]
>>> list(range(2,8))
[2, 3, 4, 5, 6, 7]
>>> list(range(0,8,1))
[0, 1, 2, 3, 4, 5, 6, 7]
>>> list(range(0,8,2))
[0, 2, 4, 6]
>>> list(range(0,8,3))
[0, 3, 6]
>>> list(range(0,8,-1))
[]
>>> list(range(8,0,-1))
[8, 7, 6, 5, 4, 3, 2, 1]
>>> list(range(8,0,-2))
[8, 6, 4, 2]
```

Doing something n times

- ```
  for i in range(3):
      print ("If I tell you three times it must be true.")
  ```

- ```
  If I tell you three times it must be true.
  If I tell you three times it must be true.
  If I tell you three times it must be true.
  ```

- ```
  for i in range(4):
      print(i, "squared is", i*i, "and cubed is", i*i*i)
  ```

- ```
  0 squared is 0 and cubed is 0
  1 squared is 1 and cubed is 1
  2 squared is 4 and cubed is 8
  3 squared is 9 and cubed is 27
  ```

Exercise

- Write a program that prints a string 10 times. Include the loop count in the output
- Expected output:
-
```
0 TACG
1 TACG
2 TACG
3 TACG
4 TACG
5 TACG
6 TACG
7 TACG
8 TACG
9 TACG
```

Exercise

- Here is a Python list of restriction site patterns
-
```
RestrictionSites = [
 "GAATTC",   #EcoRI
 "GGATCC",   #BamHI
 "AAGCTT",   #HindIII
]
```

This is a comment, this has no effect on the execution, we will come back to this

- Write a program that prints each pattern

```
GAATTC is a restriction site
GGATCC is a restriction site
AAGCTT is a restriction site
```

Exercise

- Modify the previous exercise to check whether each restriction site is present or not in a DNA sequence taken from the user
-
```
Insert a sequence:AGAATTC
GAATTC is in the sequence: True
GGATCC is in the sequence: False
AAGCTT is in the sequence: False
```

Auxiliary variables

- In many cases it is useful to have an auxiliary variable keeping track of partial results inside a loop
  - It needs to be initialized before the loop
- Example: compute the sum of the first ten squares
-
```
sum = 0
for i in range(1, 11):
   sum = sum + (i * i)
print("The sum of the first 10 squares is", sum)
```

- Output:

```
The sum of the first 10 squares is 385
```

Beware of approximations

- Computer systems represent floating point numbers with finite approximation
  - Such approximations may compose in strange ways
-
```
j=2.0
for i in range(10):
    j=j-0.2
    print(j)
```

- Which result do you expect?
- And what if we replace `0.2` with `0.25`?

Exercise

- Write a python program that computes the factorial of a number taken from the user

Exercise

- Write a python program that draws on the screen a diagonal line of symbols \ of length n
- E.g., for n=3 you get:

```
\
  \
    \
```

The `for` loop, more precisely

- Syntax:
```
for variableName in groupOfValues:
    statements
```
- **`groupOfValues`** can be a string, a list, a range, and also other kinds of data
  - we will see some other examples later on in the course
- **`variableName`** allows one to refer to the values, one at the time
- **`statements`** is any sequence of statements, including other `for` loops
  - it will be executed once for each element in **`groupOfValues`**

The `for` loop execution, more precisely

- ```
  for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
      print("Hello,", name)
  print("The end.")
  ```

Variable name	Value
  ```

Start from the first line

```
for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
    print("Hello,", name)
print("The end.")
```

Variable name	Value

Look at the list

```
for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
    print("Hello,", name)
print("The end.")
```

Variable name	Value

Is it empty? If not, start from the first element

```
for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
    print("Hello,", name)
print("The end.")
```

Variable name	Value

Assign the first element to variable `name`

```
for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
    print("Hello,", name)
print("The end.")
```

Variable name	Value
`name`	`"Andrew"`

Then execute the first line of the code block

- ```
  for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
      print("Hello,", name)
  print("The end.")
  ```

Variable name	Value
name	"Andrew"
  ```

This is the print statement

```
for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
    print("Hello,", name)
print("The end.")
```

Variable name	Value
name	"Andrew"

```
Hello, Andrew
```

The code block is finished

```
for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
    print("Hello,", name)
print("The end.")
```

Variable name	Value
name	"Andrew"

```
Hello, Andrew
```

Go back to the `for` statement

```
for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
    print("Hello,", name)
print("The end.")
```

Variable name	Value
`name`	`"Andrew"`

```
Hello, Andrew
```

Move to the next element

```
for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
    print("Hello,", name)
print("The end.")
```

Variable name	Value
name	"Andrew"

```
Hello, Andrew
```

Is it past the last element? No

- ```
  for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
      print("Hello,", name)
  print("The end.")
  ```

Variable name	Value
name	"Andrew"

```
Hello, Andrew
```
```

Is it past the last element? No

- ```
  for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
      print("Hello,", name)
  print("The end.")
  ```

Variable name	Value
name	"Andrew"

```
Hello, Andrew
```
```

Assign the second element to variable `name`

- ```
  for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
      print("Hello,", name)
  print("The end.")
  ```

Variable name	Value
`name`	`"Tsanwani"`

```
Hello, Andrew
```
```

Then execute the first line of the code block

```
for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
    print("Hello,", name)
print("The end.")
```

Variable name	Value
name	"Tsanwani"

```
Hello, Andrew
```

And the story goes on till...

```
for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
    print("Hello,", name)
print("The end.")
```

Variable name	Value
name	"Sophie"

```
Hello, Andrew
Hello, Tsanwani
Hello, Marco
Hello, Sophie
```

… the pointer goes past the last element

- ```
  for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
      print("Hello,", name)
  print("The end.")
  ```

Variable name	Value
name	"Sophie"

```
Hello, Andrew
Hello, Tsanwani
Hello, Marco
Hello, Sophie
```
```

Skip the code block and go to the next statement

```
for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
    print("Hello,", name)
print("The end.")
```

Variable name	Value
name	"Sophie"

```
Hello, Andrew
Hello, Tsanwani
Hello, Marco
Hello, Sophie
```

Execute it

```
for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
    print("Hello,", name)
print("The end.")
```

Variable name	Value
name	"Sophie"

```
Hello, Andrew
Hello, Tsanwani
Hello, Marco
Hello, Sophie
The end.
```

The program has ended, you are done

```
for name in ["Andrew", "Tsanwani", "Marco", "Sophie"]:
    print("Hello,", name)
print("The end.")
```

Variable name	Value
name	"Sophie"

```
Hello, Andrew
Hello, Tsanwani
Hello, Marco
Hello, Sophie
The end.
```

Nested loops

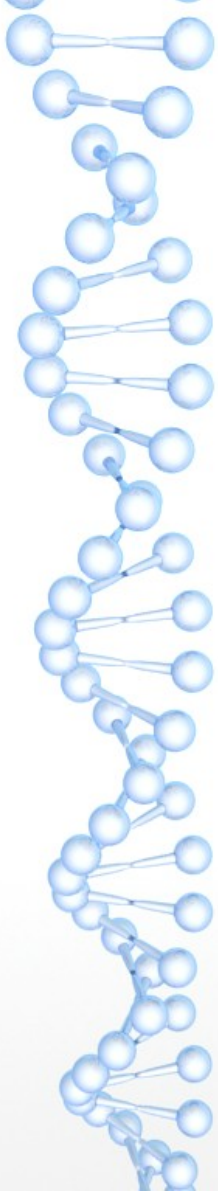
- You can put a loop inside a loop
  - All the times you want
-
```
numbers =
["A","2","3","4","5","6","7","8","9","10","J","Q","K"]
signs = ["clubs","diamonds","hearts","spades"]
for sign in signs:
    for number in numbers:
        print(number,"of",sign)
    print("-------------")
    print(len(numbers),"cards of",sign)
print("-------------")
print("-------------")
print(len(numbers)*len(signs),"cards in total")
```

Exercise

- Write a program that takes an integer n from the user and builds the list of all possible pairs of numbers [i,j] such that $0 \leq i,j < n$
- And if you want always $i < j$?
- What if you want always $i \neq j$?

Exercise

- Take from the user an integer n and print on the screen the nxn multiplication table, i.e., an nxn table where cell in position i,j (0<i,j<=n) has value i*j



If-else

Ivan Lanese

Conditionals

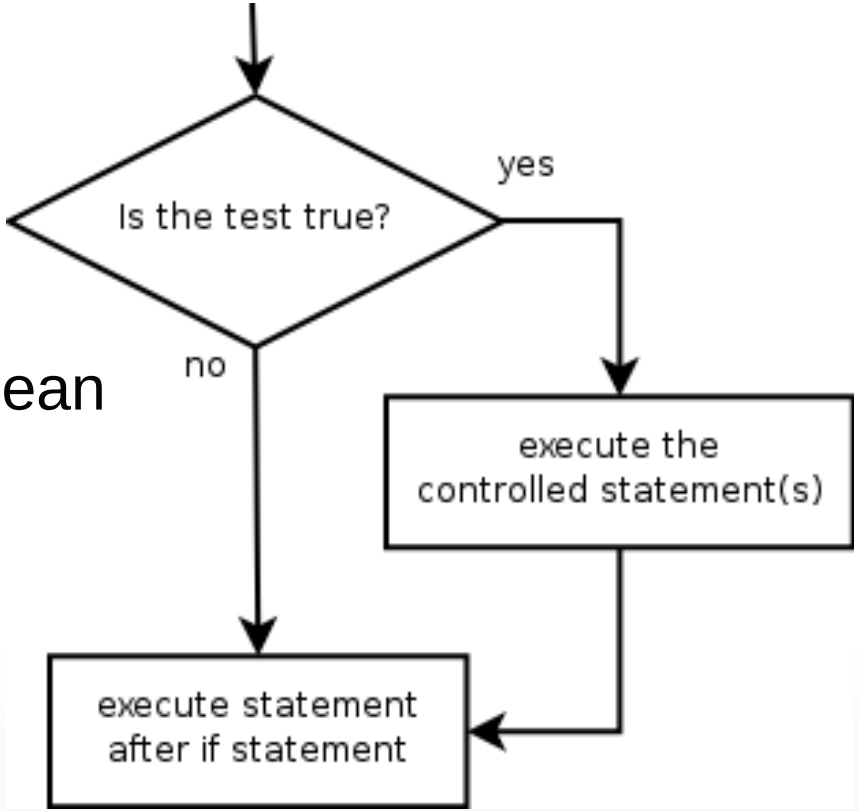
- In many cases we want programs to behave differently according to whether data have some properties, e.g.,
  - Only select data with some features (e.g., DNA sequences containing some restriction sites)
  - Only accept some form of input (e.g., DNA sequences should only contain characters ACGT)
  - ...
- The `if-else` construct tackles this need

The `if` statement

- ```
  >>> n=3
  >>> if n>o:
  ...     print(n,"is positive")
  ...
  3 is positive
  >>> n=-2
  >>> if n>o:
  ...     print(n,"is positive")
  ...
  >>>
  ```
- The code block after the `if` line can be multiline and must be indented
  ```

The `if` statement, more precisely

- Syntax:
  `if` **`condition:`**
  `statements`
- **`condition`** can be any expression producing a Boolean value (`True` or `False`)
- if **`condition`** is `True` then **`statements`** is executed, otherwise it is skipped



The `if-else` statement

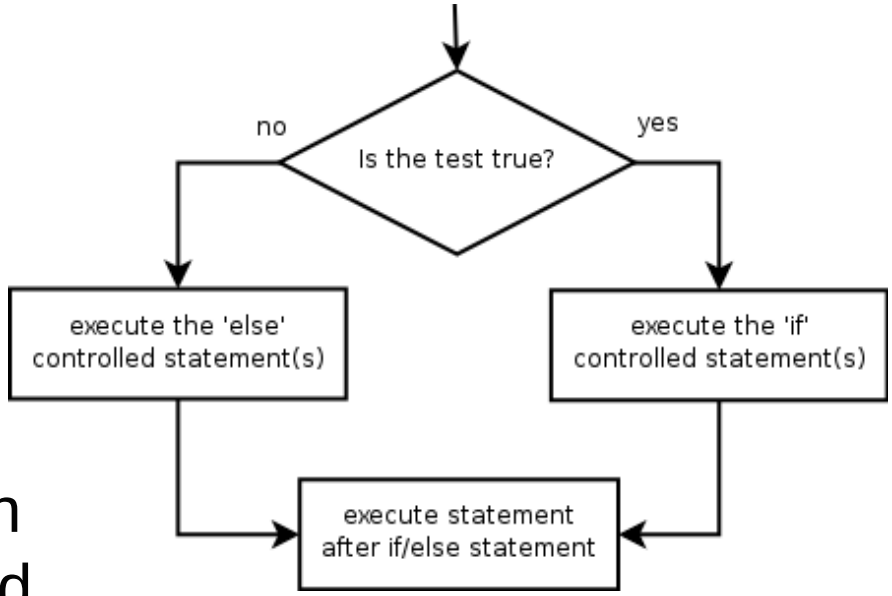
```
>>> n=3
>>> if n>o:
...     print(n,"is positive")
... else:
...     print(n,"is not positive")
...
3 is positive
>>> n=-2
>>> if n>o:
...     print(n,"is positive")
... else:
...     print(n,"is not positive")
...
-2 is not positive
>>>
```

The `if-else` statement, more precisely

- Syntax:

```
if condition:
    statements1
else:
    statements2
```

- if **`condition`** is `True` then **`statements1`** is executed, otherwise **`statements2`** is executed



The `if-elif-else` statement

```
>>> n=3
>>> if n>o:
...     print(n,"is positive")
... elif n==0:
...     print(n,"is null")
... else:
...     print(n,"is negative")
...
3 is positive
>>> n=-2
>>> if n>o:
...     print(n,"is positive")
... elif n==0:
...     print(n,"is null")
... else:
...     print(n,"is negative")
...
-2 is negative
>>>
```

The `if-elif-else` statement

```
>>> n=3
>>> if n>o:
...     print(n,"is positive")
... elif n==0:
...     print(n,"is null")
... else:
...     print(n,"is negative")
...
3 is positive
>>> n=-2
>>> if n>o:
...     print(n,"is positive")
... elif n==0:
...     print(n,"is null")
... else:
...     print(n,"is negative")
...
-2 is negative
>>>
```

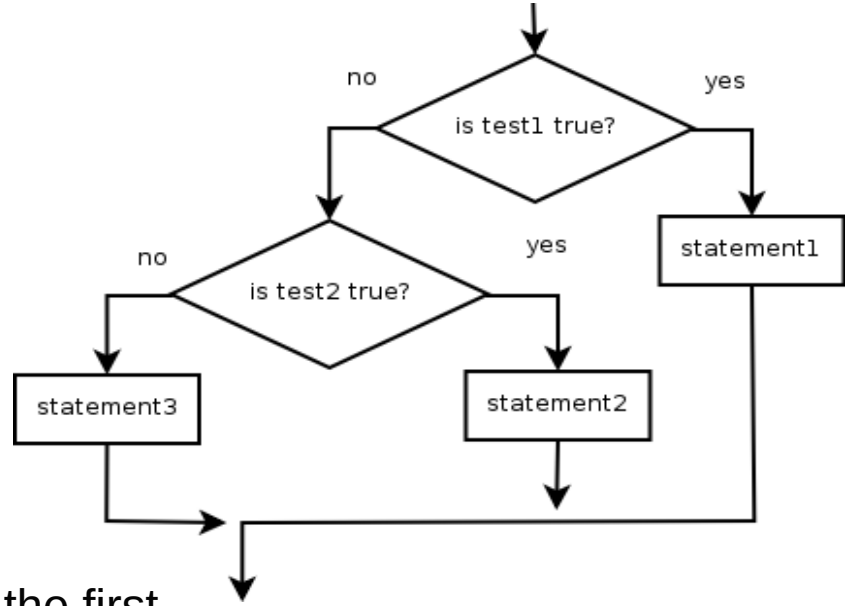
Equality test

The `if-elif-else` statement, more precisely

- Syntax:

```
if condition1:
    statements1
elif condition2:
    statements2
...
elif condition n-1:
    statements n-1
else:
    statements
```

- The statement corresponding to the first `True` condition is executed, if no condition is `True` then the `else` statement is executed



Relational operators

- Conditions are frequently built using relational operators

Operator	Meaning	Example	Result
`==`	equal	`1 + 1 == 2`	`True`
`!=`	not equal	`3.2 != 2.5`	`True`
`<`	less than	`10 < 5`	`False`
`>`	greater than	`10 > 5`	`True`
`<=`	less than or equal to	`126 <= 100`	`False`
`>=`	greater than or equal to	`5.0 >= 5.0`	`True`

Logical operators

- Logical expressions can be combined using logical operators

Operator	Example	Result
`and`	`9 != 6 and 2 < 3`	`True`
`or`	`2 == 3 or -1 < 5`	`True`
`not`	`not 7 > 0`	`False`

Example

- ```
  if "GAATTC" in s:
     print("EcoRI site is present")
  ```
- What if you want to do something if the condition is false?
- Python3 has a `not in` operator
  ```
  if "GAATTC" not in s:
     print("EcoRI will not cut the sequence")
  ```
- You can also use logical operator `not`
  ```
  if not "GAATTC" in s:
     print("EcoRI will not cut the sequence")
  ```
  ```

The example in the python3 shell

```
>>> x = True
>>> x
True
>>> not x
False
>>> not not x
True
>>> if "GAATTC" not in "AAAAAAAAA":
...     print("EcoRI will not cut the sequence")
...
EcoRI will not cut the sequence
>>> if not "GAATTC" in "ATCTGGAATTCATCG":
...     print("EcoRI will not cut the sequence")
...
>>> if not "GAATTC" in "AAAAAAAAAA":
...     print("EcoRI will not cut the sequence")
...
EcoRI will not cut the sequence
>>>
```

`else:`

- With the `else` clause you can do something if the condition is `True` and something else if it is `False`
-
```
if "GAATTC" in "ATCTGGAATTCATCG":
    print("EcoRI site is present")
else:
    print("EcoRI will not cut the sequence")
```

Where is the site?

- You can use `find` to discover the position of the site
-
```
seq = input("Insert a DNA sequence:")
pos = seq.find("GAATTC")
if pos == -1:
    print("EcoRI does not cut the sequence")
else:
    print("EcoRI site starting at index",
pos)
```

A more complex example

- You can combine `for` and `if`
-
```
restriction_sites = [
    "GAATTC",  #EcoRI
    "GGATCC",  #BamHI
    "AAGCTT",  #HindIII
]
seq=input("Insert a sequence:")
for site in restriction_sites:
    if site in seq:
        print(site,"is a cleavage site")
    else:
        print(site,"is not present")
```

Nested code blocks

```
restriction_sites = [
    "GAATTC",  #EcoRI
    "GGATCC",  #BamHI
    "AAGCTT",  #HindIII
]
seq=input("Insert a sequence:")
for site in restriction_sites:
    if site in seq:
        print(site,"is a cleavage site")
    else:
        print(site,"is not present")
```

This is the code block inside the `for` loop

Exercise

- Write a program that takes two numbers and a symbol chosen between `+`,`-`,`*`,`/` and performs the corresponding operation

Exercise

- Write a program to display and count the divisors of a given positive number
- Write a program that takes a positive number `n` and prints the prime numbers smaller than `n`

Exercise

- Write a program that prints only characters ACGT in a given string
  - E.g., `ADCFCAG → ACCAG`
- Write a program that complements a given DNA sequence. The program should print an error message if given a string which is not a DNA sequence
  - E.g, `ACCT → TGGA`

Exercise

- Write a program that removes duplicated elements from a list
  - E.g., `[2,3,4,5,4,4,6,5]` → `[2,3,4,5,6]`

Functions

Ivan Lanese

Modularity

- You now have the tools to write down a number of interesting programs
- However, when programs get bigger managing them starts being quite difficult
- We need tools to give more structure to programs
- We will see functions as an example of these tools, we will see other examples later on

You already know functions

- Indeed you have used a number of built-in functions
-
```
>>> len("ATGGTCA")
7
>>> abs(-6)
6
>>> float("3.1415")
3.1415
>>>
```
- You can also define functions yourself

What are functions?

- A function is a code block with a name

```
>>> def hello():
...     print("Hello, how are you?")
...
>>> hello()
Hello, how are you?
>>>
```

What are functions?

- Functions start with `def`

```
>>> def hello():
...     print("Hello, how are you?")
...
>>> hello()
Hello, how are you?
>>>
```

What are functions?

- Then there is the name
- This function is named `hello`

```
>>> def hello():
...     print("Hello, how are you?")
...
>>> hello()
Hello, how are you?
>>>
```

What are functions?

- Then the list of parameters
- In this case there are no parameters: the list is empty
- We will come back to parameters soon

```
>>> def hello():
...     print("Hello, how are you?")
...
>>> hello()
Hello, how are you?
>>>
```

What are functions?

- Finally the body of the function
- This can be any code block
- Indentation is needed

```
>>> def hello():
...     print("Hello, how are you?")
...
>>> hello()
Hello, how are you?
>>>
```

Invoking functions

- When you invoke a function you ask python3 to execute its body
- Functions are called by name, `hello` in this case

```
>>> def hello():
...     print("Hello, how are you?")
...
>>> hello()
Hello, how are you?
>>>
```

Invoking functions

- Then the list of parameters
- In this case there are no parameters: the list is empty

```
>>> def hello():
...     print("Hello, how are you?")
...
>>> hello()
Hello, how are you?
>>>
```

Invoking functions

- … and the function executes

```
>>> def hello():
...     print("Hello, how are you?")
...
>>> hello()
Hello, how are you?
>>>
```

Parameters

- Most of the times you do not want to execute exactly the same code again and again
- You actually want to apply the same algorithm to different data
- E.g., check whether the same restriction site occurs in many DNA sequences
- Or maybe check whether various restriction sites occur in many DNA sequences
- You can define a generic function looking whether a generic restriction site occurs in a generic DNA sequence and invoke it for specific restriction sites and DNA sequences, passed as parameters

Hello <insert name here>

- Let’s change the function hello so to take as a parameter the name of the person to greet

```
>>> def hello(name):
...     print("Hello",name)
...
>>> hello("Andrew")
Hello Andrew
>>>
```

Changes to the function declaration

- Now the function takes `name` as a (formal) parameter
- At runtime, it will refer to the value of the actual parameter

```
>>> def hello(name):
...     print("Hello",name)
...
>>> hello("Andrew")
Hello Andrew
>>>
```

Changes to the function invocation

- In the invocation, `"Andrew"` is passed as (actual) parameter
- At runtime, `name` will refer to `"Andrew"`

```
>>> def hello(name):
...     print("Hello",name)
...
>>> hello("Andrew")
Hello Andrew
>>>
```

Multiple parameters

- A function can have any number of parameters
- The correspondence between formal and actual parameters is determined by their position

```
>>> def sub(x,y):
...     print(x-y)
...
>>> sub(6,3)
3
>>>
```

Parameters in practice

- If a function needs some information to work on, 95% of the times it takes such information from a parameter
- The remaining 5% are input functions, whose aim is to take data from the user or from some other data source
  - If you have a function doing data processing taking data from the user, you are probably not doing the right thing
  - Input functions normally do no or little data processing

Returning values

- Very few functions show information to the user
  - Actually, only output functions (e.g., `print`)
- Functions normally return values, which are then assigned to variables and can be used as desired in the program
- E.g., the `len` function returns a value, which can then be used in a `range`, in a conditional, printed, ...
- If a value is printed only, it is not available for further elaboration

The `return` statement

- It is used to return a value to the caller
-
```
>>> def sub(x,y):
...     return(x-y)
...
>>> x=sub(6,3)
>>> print(x)
3
>>>
```
- Any value can be returned (numbers, strings, lists, ...)

Default return value

- If a function ends without a return value, then `None` is used as a return value
-
```
>>> def sub(x,y):
...     print(x-y)
...
>>> x=sub(6,3)
3
>>> print(x)
None
>>>
```

Functions in practice

- Normally, functions take values from parameters and return results to the caller
- (Almost) Only input functions take data from the user, (almost) only output functions provide output towards the user
- This allows to reuse functions in different contexts
  - Take data from different sources (previous elaborations, user, file, …)
  - Use the result in different ways (further elaborations, printing, save on file, …)

Structuring your code

- Good code needs to be properly structured into functions
- Broadly speaking, each “complete” functionality can be coded as a function
- If functions are invoked by your main program, they should be defined before the invocation
- It is good practice to first define all your functions, and at the end write your main program

Advantages of functions

- Functions make your code more structured and easy to understand and modify
- Functions also allow one to limit code repetitions
  - Instead of writing twice the same piece of code, write it once as a function, and invoke it twice
  - This simplifies also future updates
- Correct use of functions is fundamental to write large pieces of software
- Functions can also be delivered without a main program: libraries (e.g., NumPy)

Practicing with functions

- Functions (unless they are recursive, we will come back to this) do not allow you to write more programs
- Just allow you to write the same programs in a better way
- From now on you should try to correctly structure your programs using functions (we will come back to this)
- You can go back to past programs and structure them
- E.g., try with:
  Write a program that removes duplicated elements from a list

Exercises

- Define a function to compute the area of a triangle given base and height
- Define a function to compute the average of a list of integers

Writing readable code

Ivan Lanese

Writing readable code

- Your code may be read by others or by yourself in the future
  - Someone may want to extend or change your code
  - You may want to publish (part of) your code to describe some data processing technique
  - You may go to a different lab and someone has to take over your work
- It is not easy to understand code, not even your own code
- A number of techniques can be used to improve code readability

Docstrings

- Docstrings are a way to document functions (and other constructs we have not discussed yet)
- They describe their behaviour
- They are meant for people that want to use the function
- They are the first statement of a function definition
  - Hence they need to be indented
- They are enclosed in triple `"` and can be multiline

Sample docstrings

- ```
  def dbl(x):
      """This function takes a number x as
      input and returns 2 * x"""
      return 2 * x
  ```
- They are more useful in functions more complex than the one above
  ```

Retrieving docstrings

- You can retrieve docstrings using function `help`
- `help(dbl)` shows
- `Help on function dbl in module __main__:`

```
dbl(x)
    This function takes a number x as
    input and returns 2 * x
```

- You can do the same with builtin functions such as `len` or `sqrt`
  - For the latter, you first need to import it
  - Sometimes the result is more complex than you expect (try `range`)

Comments

- Comments are aimed at yourself or people that want to modify your code
- Comments start with `#` and go to the end of line
- `#This is a comment`
- Comments are used to clarify your code
  - Explaining tricky points
  - Clarifying the expected use of a variable
- You should not comment every line
  - E.g., the following comment is useless, even harmful
  - `i = i + 1  #increments variable i`
- Comments have no impact on the execution
  - Sometimes they are used to temporarily disable some line of code

Naming

- You should use names (of variables, functions, ...) that help understanding the code
- You should avoid names which are too short (`a`) or too long (`variable_used_to_keep_the_list_of_students`)
- Something like `students` or `student_list` is better
- By convention in python function and variable names are lowercase, with words separated by underscore (as in the examples above)

Try to write clean code

- Every problem can be solved by many different code fragments
- Try to find the cleanest one
  - Or at least close to it
- This makes it more readable, and also simplifies writing it
- Frequently, clarity is even more important than efficiency

Bad code

```
def math(z):
    v = []
    for a in range(1,z+1):
        if z%a==0:
            v.append(a)
    return(v)
```

Better code

```
def divisors(n):
    """Returns the list of the divisors
    of the integer n"""
    lst = []
    for div in range(1,n+1):
        if n%div==0: #div is a divisor if
                     #the remainder is 0
            lst.append(div)
    return(lst)
```

Exercises

- You already solved the exercises below as part of the lesson on functions. Update your solutions so to make them readable following the guidelines we discussed.
- Define a function to compute the average of a list of integers
- Write a program that removes duplicated elements from a list

The `while` loop

Ivan Lanese

Why `for` loop is not enough?

- The for loop allows you to iterate a block of code a number of times (using `range`, or iterating on a data structure)
- However, you need to know in advance how many times you want to loop, or at least an upper bound
- Sometimes you don't know
  - Reading input from the user until he requires to quit
  - Refining an approximation till the error is small enough
  - Make chess moves till checkmate
- The `while` loop is more general than the `for` loop, and solves this kind of problems

The `while` loop

- ```
  def countdown(n):
      while n > 0:
          print(n)
          n = n-1
      print("Blastoff!")
  ```

  This is the guard of the loop

- The body of the `while` loop is executed while the condition is true
- You could have done this example with a `for` loop as well, but this is not always the case
  ```

The `while` loop behaviour

1) The Boolean guard is evaluated.
2) If the condition is `False`: while ends, and execution continues with the statement after it.
3) If the condition is `True`: the body of the loop is executed, after this execution goes back to point 1).

Notes on the `while` loop

- If the guard is initially `False`, the body is never executed
- Normally, the body should contain commands that change the value of variables used in the while guard
- Otherwise we get infinite loops
  - If there are no commands that allow to exit the loop, such as `return`

A strange function

```
def sequence(n):
    while n != 1:
        print(n)
        if n%2 == 0:    # n is even
            n = n//2
        else:           # n is odd
            n = n*3+1
```

- This seems a very simple function on integers
- But does it terminate for every n?

Termination

- The value of n sometimes increases and sometimes decreases
- For some n, proving termination is very easy (e.g., if n is a power of 2)
- Termination of this function is actually an open problem, called Collatz conjecture
  - This is considered very difficult to prove (or disprove)

Termination, in general

- In general, deciding whether a function terminates for every possible input is undecidable
- That is, there exist functions for which this cannot be decided
- The function before may be one of those or not

The `break` statement

- Sometimes we may want to exit a loop before its end
-
```
def double_till(lst,threshold):
    """Doubles the elements of lst till
    one >= than the threshold is found"""
    i=0
    while(i<len(lst)):
        if lst[i]<threshold:
            lst[i]=lst[i]*2
        else:
            break
        i=i+1
    return lst
```
- You can get the same result without `break`

Notes on the `break` statement

- You can use the `break` statement in `for` loops as well
- If you use the `break` statement inside nested loops, you will only exit the innermost one
- If you are inside a function sometimes you can use `return` instead of `break`

Infinite loop with `break`

- Sometimes the guard can be `True` and the `break` can be the only exit condition
  - This may be useful, e.g., when asking the user for input
-
```
while True:
    line = input(">> ")
    if line == "EXIT":
        print("bye")
        break
    print(line)
```

Exercises

- Define a function that asks numbers to the user till (s)he enters an even number. The function returns the number of tries.
- Define a function that asks numbers to the user till (s)he enters a positive number. The function returns the square root of the last inserted number.

Exercises

- Define a function that checks whether a list of integers contains at least an even number. Write down a solution using `while` loop and a solution using for loop.

Exercises

- Rewrite the examples about `break` at slides 9 and 11 without using `break`

Functions (2nd part)

Ivan Lanese

Old exercise from python-if

- Write a program that takes two DNA sequences and prints 3 rows: the first and the third for the two sequences, the second one should contain “|” if the bases are complementary, “X” if not. Example:

```
ATTCGT
||X|X|
TAGGAA
```

A solution without functions

```
seq1 = input("Insert a DNA sequence:")
seq2 = input("Insert a 2nd DNA sequence:")
bases = "ACGT"
diff_seq = ""
for i in range(len(seq1)):
    pos1 = bases.find(seq1[i])
    pos2 = bases.find(seq2[i])
    if pos1 + pos2 == 3:
        diff_seq = diff_seq + "|"
    else:
        diff_seq = diff_seq + "X"
print(seq1)
print(diff_seq)
print(seq2)
```

Can functions help?

- Large part of the code is used to compute the string showing differences and commonalities
- This is a good candidate to become a function
- Are there candidates to become parameters or return values?

Analyzing our program

```
seq1 = input("Insert a DNA sequence:")
seq2 = input("Insert a 2nd DNA sequence:")
bases = "ACGT"
diff_seq = ""
for i in range(len(seq1)):
    pos1 = bases.find(seq1[i])
    pos2 = bases.find(seq2[i])
    if pos1 + pos2 == 3:
        diff_seq = diff_seq + "|"
    else:
        diff_seq = diff_seq + "X"
print(seq1)
print(diff_seq)
print(seq2)
```

A solution with functions

```
def find_mismatches(seq1,seq2):
    """Computes …"""
    bases = "ACGT"
    diff_seq = ""
    for i in range(len(seq1)):
        pos1 = bases.find(seq1[i])
        pos2 = bases.find(seq2[i])
        if pos1 + pos2 == 3:
            diff_seq = diff_seq + "|"
        else:
            diff_seq = diff_seq + "X"
    return(diff_seq)

seq1 = input("Insert a DNA sequence:")
seq2 = input("Insert a 2nd DNA sequence:")
print(seq1)
print(find_mismatches(seq1,seq2))
print(seq2)
```

What about `bases` and `diff_seq`?

- Why initializing them inside the function? Maybe we can initialize them outside and pass them as parameters?
- Not a good idea
- They are never used outside
- They always have the same value
- Also, changing their value will compromise the correct behaviour of the function
  - And the correct value of `bases` is also a bit tricky
- They are perfect candidates to be local variables

Scope of variables

- Variables defined inside a function are visible from their definition till the end of the function
  - They are called local variables
- They are not visible in the caller nor in other functions
- Even if the same variable name is used, it actually corresponds to different variables in different functions
- Hence, to communicate values between functions we use parameters and return values
- Variables defined outside all the functions are global and can be seen inside functions
  - It is better not to exploit this mechanism anyway

Are we done with our example?

- The comparison of nucleotides is meaningful also in other contexts
- We may want to isolate it, so to be able to reuse it

Analyzing our program, again

```
def find_mismatches(seq1,seq2):
    """Computes …"""
    bases = "ACGT"
    diff_seq = ""
    for i in range(len(seq1)):
        pos1 = bases.find(seq1[i])
        pos2 = bases.find(seq2[i])
        if pos1 + pos2 == 3:
            diff_seq = diff_seq + "|"
        else:
            diff_seq = diff_seq + "X"
    return(diff_seq)

seq1 = input("Insert a DNA sequence:")
seq2 = input("Insert a 2nd DNA sequence:")
print(seq1)
print(find_mismatches(seq1,seq2))
print(seq2)
```

A first possibility

```
def find_mismatch(nucl1,nucl2):
    bases = "ACGT"
    pos1 = bases.find(nucl1)
    pos2 = bases.find(nucl2)
    if pos1 + pos2 == 3:
        return "|"
    else:
        return "X"


def find_mismatches(se1,seq2):
    diff_seq = ""
    for i in range(len(seq1)):
        diff_seq = diff_seq + find_mismatch(seq1[i],seq2[i])
    return(diff_seq)

# Main program is unchanged
# Would be good to add docstrings
```

A second possibility

```
def find_mismatch(nucl1,nucl2):
    bases = "ACGT"
    pos1 = bases.find(nucl1)
    pos2 = bases.find(nucl2)
    return pos1 + pos2 == 3


def find_mismatches(se1,seq2):
    diff_seq = ""
    for i in range(len(seq1)):
        if find_mismatch(seq1[i],seq2[i]):
            diff_seq = diff_seq + "|"
        else:
            diff_seq = diff_seq + "X"
    return(diff_seq)

# Main program is unchanged
# Would be good to add docstrings
```

Some comments

- There may be more than one reasonable way of structuring a program into function
- Functions can be used inside other constructs
  - We used functions inside other functions and inside the guard of a conditional

Dealing with unexpected input values

- The caller can invoke a function with any kind of parameters
- A function however has expectations on which parameters it desires
  - String, integer, list, ...
  - Positive numbers, DNA sequences, …
- You should check that parameters match the expectations, if not signal the problem to the caller
  - To check the type, you can use `isinstance(var,type)`
  - E.g, `isinstance(num,int)` – other possibilities are `str`, `list`, ...
- With what you know, you should establish a convention to return the error
  - Returning a value which is not among the expected results

Bonus stage: higher-order functions

- You may notice that function `find_mismatches` now is a generic function which iterates a character comparison on all the elements of a pair of strings
- You can change the function `find_mismatch` to compare according to different predicates
- You can actually do better: you can pass a function to be used for comparison
- Indeed, functions can be passed as parameters (as well as returned as return values)
- A function taking or returning functions is called an higher-order function

Higher-order functions: example

```
def find_mismatch(nucl1,nucl2):
   bases = "ACGT"
   pos1 = bases.find(nucl1)
   pos2 = bases.find(nucl2)
   return pos1 + pos2 == 3


def equal(x,y):
   return x==y


def find_mismatches(se1,seq2,comp):
   diff_seq = ""
   for i in range(len(seq1)):
       if comp(seq1[i],seq2[i]):
           diff_seq = diff_seq + "|"
       else:
            diff_seq = diff_seq + "X"
   return(diff_seq)
```

```
seq1 = input("Insert a DNA sequence:")
seq2 = input("Insert a 2nd DNA
sequence:")
print(seq1)
print(find_mismatches(seq1,seq2,find_mism
atch))
print(find_mismatches(seq1,seq2,equal))
print(seq2)


# Would be good to add docstrings
```

Exercises

- Write the second possibility of the complement exercise keeping into account possible unexpected inputs:
  - sequences of different lengths
  - strings which are not DNA sequences

Exercises

- Write a program that takes a list of DNA sequences and complements all of them
- Modify the previous function so that it takes a list of strings, and complements only those strings in the list which are DNA sequences
- Of course, use functions to structure your solution

Exercises: bonus stage

- Write a program that allows two players to play tic-tac-toe

Debugging

Ivan Lanese

Usually programs are wrong

```
prod = 1
n = 7 #fake value to ensure the while
      #loop is executed at least once
while(n!=0):
    n = int(input("Insert a number (0
    to exit):"))
    prod = prod * n
print("The product is",prod)
```

What does it mean that a program is correct?

- It means that for any input(s) it will provide the expected output(s)
  - A program that runs forever without interacting with the user is not correct
  - A program that crashes if the user inserts wrong values is not correct
    - In such a case the program should produce a suitable error message
- A program that sometimes crashes or produces wrong results is not correct
  - The sentence “I tried it and it was working” is not a valid claim of program correctness

Testing and verification

- When you write a program you need to test it on many possible inputs
- Testing: activity to evaluate the quality of software
- May lead to find (and then fix) bugs
- Cannot ensure software correctness
  - It is not possible to test a program on all possible inputs
  - They are normally a very high number
- To ensure software correctness one needs to do verification
  - Mathematical proof of program correctness
  - Very complex and costly
  - It is only done for small and safety-critical pieces of software

3 kinds of errors

- We will focus on functional errors
  - The program does not do what it is expected to do
- Non functional issues (performance, scalability, usability, ...) require dedicated skills you do not have (yet)
- There are 3 kinds of functional errors
  - Parsing errors
  - Runtime errors
  - Logic errors

Parsing errors

- Examples: syntax error, indentation error, …
- Notified before your program is started
- Very easy to know that there is an error, and where it is
  - Python3 provides you information
- Read the error message!
- Not always easy to fix
  - The error message may not be precise or clear
  - One cannot run the program to get further information

Runtime errors

- Examples: name is not defined, type error, division by zero, index out of range, …
- Very common in python
- They are notified during program execution, and cause its interruption
- They may not occur in every execution
- Easy to know that there is an error (if it occurs)
  - Python3 provides you information
- Read the error message!
- Not always easy to find and fix
  - Under which conditions the error actually occurs?
  - Which construct actually causes the error?
    - You had a division by 0 at line 10, where you compute x/y, but why y was 0 here?

Logic errors

- The program gives no error, but the result is not the expected one
- E.g., the result is always 0, or you expect 10 and you get 10.5
- Not even easy to know that there has been an error
  - The result may be close to the correct one
  - You may not know the correct one
  - The error may not occur in every execution
- May be very difficult to find and fix
  - Under which conditions the error actually occurs?
  - Which intermediate results are correct and which are wrong?

Debugging techniques: manual debugging

- Adding auxiliary prints to understand:
  - The flow of the execution
    - “I am taking the if branch”
    - “I am looping”
  - The intermediate states, that is the value of variables
- Always possible, since no tool support is required
- One should be careful not to introduce further errors
- May require a lot of work in large programs

Debugging techniques: debugger

- Requires to use a tool called debugger
- Many debuggers for python exist, some of them integrated in an IDE
- There is also a python library dbg to help with manual debugging, but it is not very easy to use hence we will skip it
- With a debugger you can easily see:
  - The flow of the execution: tracing, breakpoints
  - The intermediate states
- Very powerful
- We will see thonny and spyder

Thonny

- IDE for python coding, debugging and code animation
- Targets students and teachers
  - Can be good for now, not when you will be more expert
- Available for all programming platforms and included in many python installations
  - You can find more information on how to install it on your system in its website https://thonny.org/

Thonny features

- You can edit, run and debug your programs
- During debugging you can:
  - Execute the code line by line (step over – F6)
  - Go inside statements and functions execution (step into – F7)
  - Exit from function execution (step out – F8)
  - Go to a specific line (run to cursor)
  - Show the value of variables (activate it from view menu)
- There are a few other interesting possibilities, we will come back to them when we will need them

Thonny behaviour

- If you step into a statement, a small panel opens where you see how the statement is evaluated
- When you call a function a window open with the function running environment
- When you exit from a function you will go back to the previous window

Spyder

- Powerful IDE for python coding, testing, debugging and management
- Targets scientific programmers and engineers
  - Much more functionalities than you need, we will see few of them
- Available for all programming platforms
  - You can find more information on how to install it on your system in its website https://www.spyder-ide.org/

Spyder: tracing

- You can execute your program with *Debug* from menu *Debug*
- You can then execute step-by-step using *Step* from menu *Debug*
- You can restart the execution with *Continue* from menu *Debug*
- A breakpoint is a point in the program where execution halts
- You can create breakpoints with *Select/clear breakpoint* from menu *Debug*
- The execution will stop if a breakpoint is reached
- On the right you can use the *Variable explorer* to check the value of variables
- If you want to start a new debugging session use *Restart kernel* from menu *Consoles*

Spyder: debugging functions

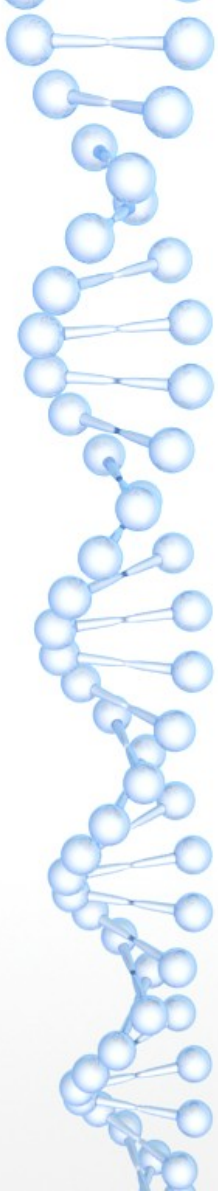
- Most programs are made of functions
- To debug them, it is better to first check single functions in isolation (divide et impera)
  - This is called unit testing
  - Define a simple main to test them with different arguments
  - Test also “unexpected” values
- When you trust single functions you can test the whole program
  - This is called integration testing
- If you execute functions with *Step*, they are executed as a single statement, if you do it with *Step into* they are executed step-by-step
- Execution using *Step* breaks anyway if there is a breakpoint inside the function

Final comments

- Debuggers are a tool, they can help but they will not find the bugs for you
- You need to reason about your code
- Please keep in mind that you should test your programs on many possible values
  - Trying to go through the various paths in your programs
  - Testing limit case (0, empty list, last element, …)
  - Testing wrong inputs (the program should not crash)
- A good way to reduce the number of errors is to write code as simple and as clean as possible

Exercise

- Each of the programs `sum_even.py` and `pairs.py` in the teaching material contains one parsing error, one runtime error and one logic error: find and fix them



Files

Ivan Lanese

Where do your data come from?

- Till now data were either hardwired in the program, or taken from the user
  - Not practical, in particular if you have large amount of data
- In most cases, data are on your computer in a dedicated file
- We need to learn how to read from and write to files

Opening a file

- In order to work with a file, you need to `open` it
- It returns an object of type `file`
- Printing it is not very useful, but there are a number of ways of working with it
-
```
>>> infile = open("./10_sequences.seq","r")
>>> print(infile)
<_io.TextIOWrapper
name='./10_sequences.seq' mode='r'
encoding='UTF-8'>
>>>
```

The `open` function

- `file = open(filename,modality)`
- `filename` is a string containing the name of the file
  - It may include the path, if not the file is looked for in the current directory
  - Under windows, remember to escape the `\`
- `modality` is a string that can have the following values
  - `"r"` for reading
  - `"w"` for writing (if file exists it is overwritten – pay attention!)
  - `"a"` for appending (if file exists data are added at the end)
- `file` will get the file handler
- Errors may be thrown (file not found, permission denied, ...)

Reading a file

- `line=file.readline()` reads the next line on the file
  - A line ends on `"\n"`
  - The `"\n"` is read as well
  - You can remove the `"\n"` with `line=line.rstrip()`
  - The last line may not have `"\n"`
- It automatically moves to the next line
- When the file is ended it returns an empty string
- You can iterate to read the whole file

Reading a file using `for`

- An alternative way to read a file is using a `for` loop
-
```
for line in file:
    #do something with line
```
- It automatically iterates on all the lines

Closing a file

- It is good policy to close files when no more needed
- Particularly important if the file is for writing
  - Otherwise some of the written strings may be lost
- You can do it with `file.close()`

Writing to file

- You can also write to file
- (An alternative is to write to screen and using Linux output redirection >)
- It must be opened using `"w"` or `"a"` modality
- `file.write(string)` writes `string` on `file`

Writing example

- ```
  outfile=open("prova.txt","w")
  outfile.write("Ciao")
  outfile.write("A tutti")
  outfile.close()
  ```
- At the end the file will contain
  `CiaoA tutti`
- No newline nor spaces are added
  ```

Command line argument

- When you run a program from the command line, you can pass it parameters
  - E.g., `cd ~/downloads`
  - `~/downloads` is a parameter
- You can do the same for python programs
  - `python3 copy.py -w myfile.txt myfile2.txt`
  - `-w`, `myfile.txt` and `myfile2.txt` are 3 parameters
- Those arguments are visible from the program, as the list `argv` from module `sys`
  - The first element is the program name, the others are the parameters
  - You need to write: from `sys` import `argv`

Exercise

- Write a program that prints `argv` on the screen. Experiment what you get if you use wildcards in parameters (ex. `*.txt, es?.py`)

  (In Windows the behaviour is much less interesting)

Exercises

- Write a program that takes a filename from the command line and prints it on the screen, with lines numbered.

```
1 …
2 …
3 …
```

- Write a program that copies a file. The names of the source and of the destination need to be taken from the command line. Provide a solution based on `for` loop and a solution based on `while` loop.

Exercises

- Write a program that takes two filenames from the command line and writes on the second one the lengths of the lines in the first one.
- List the sequences in file `10_sequences.seq` that contains the pattern `CTATA`. For each of them list also the position in the sequence of the first occurrence. You should find two such sequences.

Exercises

- Consider file `sequences.seq`. Answer the following questions.

A) How many sequences are in the file?

B) How many contain the pattern `CTATA`?

C) How many have more than 1000 bases?

D) How many have over 50% GC composition?

E) How many have more than 2000 bases and more than 50% GC composition?

Some more exercises

Ivan Lanese

Exercises

- Consider again file `sequences.seq`. Did your programs assumed that the sequences only contain ACGT bases? If so, update them to avoid this assumption.
- Count the number of sequences that contain some other letter

A small case study

- You are trying to identify a DNA sequence. From experiments you believe your DNA sequence has a molecular weight between 224245 and 226940. You think it might be in the database sequences.seq.
- For each sequence in that file which have a molecular weight in that range, print the molecular weight and the sequence.
- You can find a table of weights for single elements in file weights.txt

Is the answer correct?

- You would like to check whether the answer is correct, but you cannot do the computation by hand to make a comparison.
- You may compare two different programs solving the same problem (e.g., yours against one from your colleagues)
- You may test the program on files for which you know the answer
  - A smaller file for which you can compute the answer by hand
  - Files for which the answer is available online

References

Ivan Lanese

A more precise look to variables

- We said since the beginning that variables contain values
- This is not 100% precise
- Actually, variables contain references to memory locations, and memory locations contain values
- You can easily see this in Thonny, by activating the option *Heap* from menu *View*

Sharing

- Can we see some effect of references in our programs?
- Yes, since it may happen that two references refer the same location
- Hence, changing the value via one reference impacts the other reference as well
-
```
l = [1,2,3]
l1=l #new reference to the same list
l1[1]=5
print(l) #l has changed
```

When is sharing created?

- You create sharing only when you assign variable to variable
- If you assign an expression to a variable, no sharing occurs (a new value is created)
-
```
l = [1,2,3]
l1=l #new reference to the same list
l2=l[:] #an expression, no sharing
l[1]=5
print(l1) #l1 has changed
print(l2) #l2 has not changed
```

Sharing and functions

- When you invoke a function, the references are passed as parameters
  - If you invoke using an expression as actual parameter, the reference to it is created
- Hence, if you update the referenced object, changes are visible to the caller
- If instead you change the reference (or passed an expression), the caller is unaffected
-

```
def change(la,lb):
    la[1]=5
    lb=[3,4,5]


l1 = [1,2,3]
l2 = [1,2,3]
change(l1,l2)
print(l1) #l1 has changed
print(l2) #l2 has not changed
```

Removing references

- The function `del` removes references
- After `del a`, `a` is no more defined
- Accessing `a` leads to a `name 'a' is not defined` error
- However, further references to the same value are unaffected
- When there are no more references to a value it is unaccessible (and will be removed at some point)

Why references?

- It may seem that references are not very useful and somehow complex
- However, they are fundamental
- Lists may contain a lot of information, hence copying a (large) list is very time consuming
- Copying a reference is instead very fast (the time does not depend on the length of the list)
- Without references, python would be far less efficient

Tuples

- Python has another data type: tuples
- Tuples are similar to lists, but are immutable
- Tuples are denoted using parentheses, which can also be dropped
-
```
>>> date = ("Tuesday",22,"December",2020)
>>> print(date[2])
December
>>> date[1]=23
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item
assignment
>>> print(len(date))
4
>>>
```

Why tuples?

- You can solve all your computing problems using lists instead of tuples
- But sometimes tuples are handy
- Being sure that tuples cannot be modified may help
  - E.g., we will see that tuples are suitable to act as dictionary keys
- Also, the fact that parentheses can be dropped allows for some nice idiomatic forms

Multiple assignment and multiple return

- You can perform a multiple assignment
-
```
>>> d,m,y=22,12,2020
>>> print(d)
22
```
- Actually, you are assigning the tuple `(22,12,2020)` to the tuple of variables `(d,m,y)`
- You can have a function returning a tuple and use a multiple assignment to assign all the components to different variables
- `x,y = move_up(x0,y0,d)`

Unbounded number of parameters

- In function definition, `*var` stands for any number of parameters
- It can be used only as last parameter
- In this case, `var` is a tuple and will contain all the remaining parameters
- This is how functions such as `print` are defined
- We can define a function that evaluates a polynomial in `x` with the following declaration
- `def pol(x,*coeff):`
- If we invoke it with `pol(1,1,2,3)` then `coeff` will be the tuple `(1,2,3)`, and it will evaluate $1+2x+3x^2$ for $x=1$ (expected result `6`)

Exercises

- Define a function that takes any number of numeric arguments and computes their average
  - Call it as `avg = aver(1,2,4,5,6)`
- Define a function that takes any number of numeric arguments and returns their sum and their product
  - Call it as `sum,prod = sumprod(1,3,4,5,6)`
- Provide the definition for function `pol` described in the previous slide

Recursion

Ivan Lanese

Recursive functions

- Functions can call other functions
- A function can also call itself
- This is called recursion
- This is a very powerful technique, but also a bit tricky

Function calls, a recap

```
def h(x):
    if x%2 == 1:
        return f(x)+x
    else:
        return f(f(x))


def f(x):
    x = x-1
    return g(x)+1

def g(x):
    return x*2
```

```
h(3):
    return f(3)+3
```

8

5

```
f(3):
    return g(2)+1
```

4

```
g(2):
    return 4
```

Function calls, a recap

- Upon return, the execution returns just after where the function was called
- Each function invocation has its own value for local variables and parameters
- These are key points to understand recursive functions

Factorial, iterative definition

- n! = n*(n-1)*(n-2)*…*1

- ```
  def factorial(n):
      result = 1
      for k in range(1, n+1):
          result = result * k
      return result
  ```
  ```

Factorial, inductive definition

- n! = n*(n-1)*(n-2)*…*1

- n! = n*(n-1)! inductive case
  0! = 1 base case

From math induction to CS recursion

- n! = n*(n-1)! inductive case
  0! = 1 base case

- ```
  def factorial(n):
      if n == 0:
          return 1
      else:
          return n * factorial(n-1)
  ```
  ```

How recursion works?

6

- factorial(3):
    return 3*factorial(2)

2

factorial(2):
    return 2*factorial(1)

1

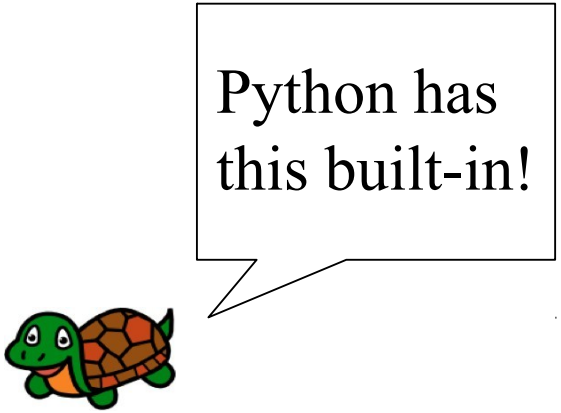
factorial(1):
    return 1*factorial(0)

1

factorial(0):
    return(1)

Power

- ```
  >>> pow(10,3)
  1000
  >>> pow(10,0)
  1
  >>> pow(2,5)
  32
  ```




- ```
  def pow(base, exp):
      if exp == 0: return 1     #base case
      else: return base * pow(base, exp-1)
  ```

General guidelines

- Base case defines the result directly in simple cases
- Inductive case defines the result in the general cases based on the result in simpler cases
- Pay attention that you always need to reach the base cases

Computing the length of a list

- ```
  >>> len([1,43,"Hi"])
  3
  >>> len([1,[2,[3,4]]])
  2
  ```

Python has this built-in too!

- ```
  def len(l):
     """Returns the length of list l"""
     if l == []: return 0
     else: return 1+len(l[1:])
  ```

Recursion may save variables

- E.g., in function `len` definition

- ```
  def len(l):
     """Returns the length of list l"""
     if l == []: return 0
     else: return 1+len(l[1:])
  ```

- ```
  def len(l):
     """Returns the length of list l"""
     counter = 0        # a new variable
     for x in l:
        counter = counter + 1
     return counter
  ```

Reversing a list

- ```
  >>> reverse([1,2,3,4])
  [4,3,2,1]
  ```

- ```
  def reverse(l):
      """Returns the reverse of list l"""
      if l == []: return []
      else: return reverse(l[1:])+[l[0]]
  ```

Computing the minimum of a list

- ```
  >>> min([34,23,56,45])
  23
  >>> min([34])
  34
  ```
- We assume the list has at least one element and we use `len` as a helper function
- ```
  def min(l):
     """Returns the smallest value in list l"""
     if len(l) == 1: return l[0]
     else:
        auxmin=min(l[1:])
        if l[0]<auxmin: return l[0]
        else: return auxmin
  ```

Checking membership

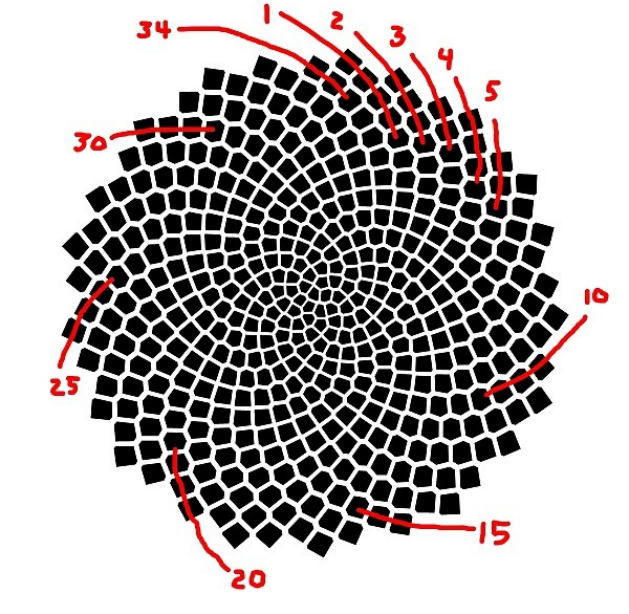
- ```
  >>> member(42,[3,45,67])
  False
  >>> member(42,[23,42,12])
  True
  ```
- Similar to python3 `in`, but defined as a function
- ```
  def member(thing,l):
      """Returns True if thing is in l, False
      otherwise"""
      if l == []: return False
      elif l[0]==thing: return True
      else: return member(thing,l[1:])
  ```

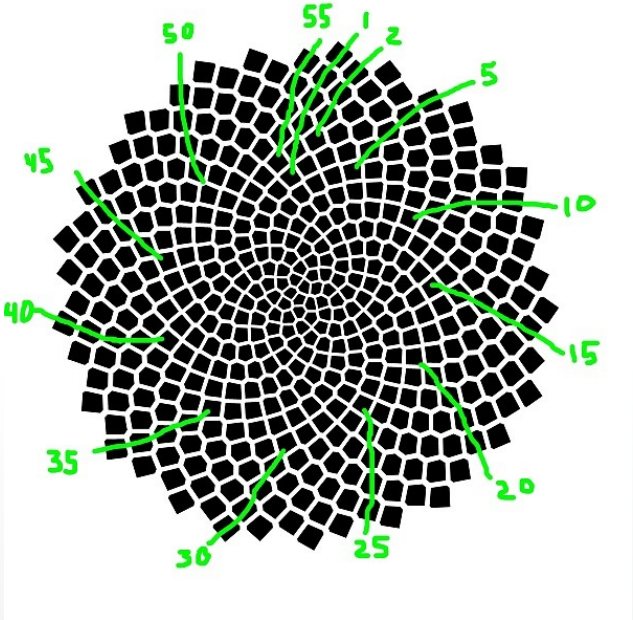
Fibonacci numbers

- Fibonacci numbers are `0,1,1,2,3,5,8,13,21,…`
- They occur in many places in nature
- They are defined by the following definition
-
```
fib(0)=0
fib(1)=1
fib(n)=fib(n-1)+fib(n-2)
```
- This can be easily translated into the following recursive definition
-
```
def fib(n):
   """Computes the n-th number of Fibonacci sequence"""
   if n == 0: return 0
   elif n == 1: return 1
   else: return fib(n-1)+fib(n-2)
```

Fibonacci numbers in nature

- Numbers of clockwise and anticlockwise spirals in sunflowers, pineapples and other vegetables tend to assume consecutive values in the Fibonacci series
- Many other examples exist





Fibonacci numbers: some notes

- In Fibonacci numbers you need two base cases, in general a recursive function can have any number of base cases (but they normally are very few)
  - You need to always reach one base case
- Recursive Fibonacci definition is not very efficient, there is an iterative definition which is less direct but far more efficient
- I think this is a good motivation for the course of Algorithms and Data Structures (and to study dynamic programming)

Final remarks on recursion

- Any function can be defined both using recursion and using a while loop
- It may be the case that one of the two definitions is much simpler than the other
  - Some recursive functions need an auxiliary data structure (a stack, that is a special case of list) to be written in iterative form
  - This is (almost) the case for Fibonacci function
- Recursion is (in many cases) the best tool to define functions on trees (e.g., the tree of directories in the file system)

A final challenge

- Do you know this game?

A final challenge

- This is called minesweeper and was a staple on Windows
- You can easily find it, e.g., at: https://minesweeper.online/
- When you click on a cell with no mines closeby, all the adjacent cells are opened, and so on
- The only reasonable way to do it is using a recursive function
- If you want, you may implement your own version of minesweeper

Exercises

- Code in python3 the function `f` defined by:
  `f(0)=3`
  `f(n)=f(n-1)*3+1`
  Execute it in thonny to understand what happens
- Define a recursive function `sum` to compute the sum of the elements of a list of numbers (analogous to python3 `sum` function)
- Define a recursive function that takes a string and returns `True` if it is palindrome, `False` otherwise

Exercises

- Define a recursive function `filter_even` that takes a list of integers and returns the list of the even numbers in it.
- Update the function `sum` in the previous slide so that it computes the sums of all the numbers in a structure where each element can be either a number or a list (which contains again numbers and lists)
- Define a recursive function `flatten` that takes a nested list as above and computes the list of its elements

Dictionaries

Ivan Lanese

A “good morning” dictionary

- English: Good morning
  Swedish: God morgon
  German: Guten morgen
  French: Bonjour
  Italian: Buon giorno

What is a dictionary?

- A dictionary is a set of items
- Each item has a key and a value
- Keys are all distinct

Key	Value
English	Good morning
Swedish	God morgon
German	Guten morgen
French	Bonjour
Italian	Buon giorno

Look up a value

- I want to know how to say “Good morning” in Swedish
- Step 1: get the “Good morning” table

Key	Value
English	Good morning
Swedish	God morgon
German	Guten morgen
French	Bonjour
Italian	Buon giorno

Look up a value

- I want to know how to say “Good morning” in Swedish
- Step 2: get the row whose key is “Swedish”

Key	Value
English	Good morning
Swedish	God morgon
German	Guten morgen
French	Bonjour
Italian	Buon giorno

Look up a value

- I want to know how to say “Good morning” in Swedish
- Step 3: the corresponding value is the answer

Key	Value
English	Good morning
Swedish	God morgon
German	Guten morgen
French	Bonjour
Italian	Buon giorno

“Good morning” dictionary in Python

```
>>> good_morning_dict = {
...     "English": "Good morning",
...     "Swedish": "God morgon",
...     "German": "Guten morgen",
...     "French": "Bonjour",
...     "Italian": "Buon giorno"
... }
>>> print(good_morning_dict["Swedish"])
God morgon
```

Some more examples

```
>>> D = {"name": "Sara", "city": "Bologna"}
>>> D["city"] = "Johannesburg"
>>> print(D)
{'name': 'Sara', 'city': 'Johannesburg'}
>>> del D["name"]
>>> print(D)
{'city': 'Johannesburg'}
>>> D["name"]="Dan"
>>> print(D)
{'city': 'Johannesburg', 'name': 'Dan'}
>>> D.clear()
>>> print(D)
{}
```

Some general notions about dictionaries

- They provide a very good way to associate values to keys
- Values can be of any type
- Keys can be of any immutable type
  - Strings, integers and tuples are good…
  - … but lists are not…
  - … and tuples containing lists are not...

DNA ambiguity codes

- Sometimes DNA bases are ambiguous.
- E.g., the sequencer might be able to tell that a base is not a G or T but could be either A or C
- The standard (IUPAC) one-letter code for DNA includes letters for ambiguous bases
- M is A or C Y is C or T D is A, G or T
  R is A or G K is G or T B is C, G or T
  W is A or T V is A, C or G N is G, A, T or C
  S is C or G H is A, C or T
- We want to count bases, including ambiguity codes

Count bases #1

```
>>> seq = "TKKAMRCRAATARKWC"
>>> A = seq.count("A")
>>> B = seq.count("B")
>>> C = seq.count("C")
>>> D = seq.count("D")
>>> G = seq.count("G")
>>> H = seq.count("H")
>>> K = seq.count("K")
>>> M = seq.count("M")
>>> N = seq.count("N")
>>> R = seq.count("R")
>>> S = seq.count("S")
>>> T = seq.count("T")
>>> V = seq.count("V")
>>> W = seq.count("W")
>>> Y = seq.count("Y")
>>> print("A =", A, "B =", B, "C =", C, "D =", D, "G =", G, "H =",
H, "K =", K, "M =", M, "N =", N, "R =", R, "S =", S, "T =", T, "V
=", V, "W =", W, "Y =", Y)
A = 4 B = 0 C = 2 D = 0 G = 0 H = 0 K = 3 M = 1 N = 0 R = 3 S = 0 T
= 2 V = 0 W = 1 Y = 0
```

Don’t do this!
Let the computer help more!

Count bases #2: using a dictionary

```
>>> seq = "TKKAMRCRAATARKWC"
>>> counts = {}
>>> counts["A"] = seq.count("A")
>>> counts["B"] = seq.count("B")
>>> counts["C"] = seq.count("C")
>>> counts["D"] = seq.count("D")
>>> counts["G"] = seq.count("G")
>>> counts["H"] = seq.count("H")
>>> counts["K"] = seq.count("K")
>>> counts["M"] = seq.count("M")
>>> counts["N"] = seq.count("N")
>>> counts["R"] = seq.count("R")
>>> counts["S"] = seq.count("S")
>>> counts["T"] = seq.count("T")
>>> counts["V"] = seq.count("V")
>>> counts["W"] = seq.count("W")
>>> counts["Y"] = seq.count("Y")
>>> print(counts)
{'A': 4, 'B': 0, 'C': 2, 'D': 0, 'G': 0, 'H': 0, 'K': 3, 'M': 1, 'N': 0, 'R':
3, 'S': 0, 'T': 2, 'V': 0, 'W': 1, 'Y': 0}
```

Don’t do this either!

Count bases #3: using a `for` loop

```
>>> seq = "TKKAMRCRAATARKWC"
>>> counts = {}
>>> for letter in "ABCDGHKMNRSTVWY":
...     counts[letter] = seq.count(letter)
...
>>> print(counts)
{'A': 4, 'B': 0, 'C': 2, 'D': 0, 'G': 0, 'H': 0, 'K': 3, 'M': 1, 'N':
0, 'R': 3, 'S': 0, 'T': 2, 'V': 0, 'W': 1, 'Y': 0}
>>> for base in counts.keys():
...     print(base,"=",counts[base])
...
A = 4
B = 0
C = 2
…
V = 0
W = 1
Y = 0
```

skipping some for space reasons

Count bases #4: discovering which bases exist

```
>>> seq = "TKKAMRCRAATARKWC"
>>> counts = {}
>>> for base in seq:
...     if base not in counts:
...             counts[base]=1
...     else:
...             counts[base]=counts[base]+1
...
>>> print(counts)
{'T': 2, 'K': 3, 'A': 4, 'M': 1, 'R': 3, 'C': 2,
'W': 1}
```

Checks whether we have found a new base

Count bases #5: using `get` with default

```
>>> seq = "TKKAMRCRAATARKWC"
>>> counts = {}
>>> for base in seq:
...     counts[base]=counts.get(base,0)+1
...
>>> print(counts)
{'T': 2, 'K': 3, 'A': 4, 'M': 1, 'R': 3,
'C': 2, 'W': 1}
```

The `get` method

- The `get` method is similar to element access, but you can provide a default to be used if the element is not found
-
```
>>> print(counts)
{'T': 2, 'K': 3, 'A': 4, 'M': 1, 'R': 3, 'C': 2, 'W': 1}
>>> print(counts["K"])
3
>>> print(counts.get("K"))
3
>>> print(counts.get("K",0))
3
>>> print(counts["N"])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'N'
>>> print(counts.get("N"))
None
>>> print(counts.get("N",0))
0
```

Complementing a sequence

- We can exploit a dictionary to provide a nice solution to the problem of complementing a DNA sequence
- ```
  comp_table = {"A": "T", "T": "A", "C": "G", "G": "C"}
  seq = input("Insert a sequence:")
  comp_seq = ""
  for letter in seq:
    comp_seq=comp_seq + comp_table.get(letter,"?")
  print("The complement is:",comp_seq)
  ```
- We put `"?"` as a complement for an unknown base
  ```

Listing codons

- We want to split a DNA sequence into codons
- ```
  seq=input("Insert a DNA sequence:")
  for i in range(0, len(seq), 3):
      print("Codon", i//3, "is", seq[i:i+3])
  ```
- A possible execution:
- ```
  Insert a DNA sequence:AGTCGTAT
  Codon 0 is AGT
  Codon 1 is CGT
  Codon 2 is AT
  ```
- The last item is not a codon, what to do?

The last “codon”

- Solution 1: refuse to process sequences whose length is not divisible by 3
- ```
  if len(seq)%3 != 0:
      print("Sequence length not divisible by 3")
  else:
       #rest of the program
  ```
- Solution 2: skip the last item
- ```
  for i in range(0, len(seq)-len(seq)%3, 3):
      print("Codon", i//3, "is", seq[i:i+3])
  ```
- Solution 1 is probably a bit too harsh

Counting codons

- We can use the same approach we used for bases to count codons as well
- ```
  seq=input("Insert a DNA sequence:")
  codon_counts = {}
  for i in range(0, len(seq)-len(seq)%3, 3):
      codon = seq[i:i+3]
      codon_counts[codon] = codon_counts.get(codon, 0)+1
  print(codon_counts)
  ```
- Sample execution:
- ```
  Insert a DNA sequence:TCTCCAAGACGCATCCCAGTG
  {'TCT': 1, 'CCA': 2, 'AGA': 1, 'CGC': 1, 'ATC': 1,
  'GTG': 1}
  ```
- Note that the elements are not sorted

Sorting the output

- We can use `keys()` to get the keys and `sort()` to sort them
-
```
seq=input("Insert a DNA sequence:")
codon_counts = {}
for i in range(0, len(seq)-len(seq)%3, 3):
    codon = seq[i:i+3]
    codon_counts[codon] = codon_counts.get(codon, 0) + 1
codons = list(codon_counts.keys())  #conversion to list needed: keys()
codons.sort()                       #produces a strange thing
for codon in codons:
    print(codon, "=", codon_counts[codon])
```
- Sample execution:
-
```
Insert a DNA sequence:TCTCCAAGACGCATCCCAGTG
AGA = 1
ATC = 1
CCA = 2
CGC = 1
GTG = 1
TCT = 1
```

Exercise

- Update the case study about molecular weight to use a dictionary. This should simplify the solution.

Exercise

- Write a program to count the total number of bases in all of the sequences in a file and the total number of each base found, in order.
-
```
File has 1179412 bases
A = 319094
B = 2
C = 242701
D = 9
G = 273117
H = 4
K = 12
M = 23
N = 3478
R = 34
S = 13
T = 340876
V = 1
W = 15
Y = 33
```

Exercise

- You can ask python for the current time using the `datetime` library
- `from datetime import datetime`
-
```
start_time = datetime.now()
#code
end_time = datetime.now()
print("Time taken:",end_time-start_time)
```
- Check how much time you need to count bases on `sequences.seq`

Exercise

- Write a program that complements all sequences in a given file, writing the result on another file
- The program should work also on ambiguous codes, using the dictionary below
-
```
ambiguous_dna_complement = {
"A": "T",
"C": "G",
"G": "C",
"T": "A",
"M": "K",
"R": "Y",
"W": "W",
"S": "S",
"Y": "R",
"K": "M",
"V": "B",
"H": "D",
"D": "H",
"B": "V",
"N": "N",
}
```

Exercise: translate DNA into protein

- Write a program that translates a given DNA sequence into protein. You can find the codon table to use on the Virtuale website. When the codon doesn't code for anything (e.g., stop codon), use "*". Ignore the extra bases if the sequence length is not a multiple of 3. Decide how you want to handle ambiguous codes.

Regular expressions

Ivan Lanese

Regular expressions

- We already discussed different ways of finding subsequences inside strings
- Sometimes one is not looking for a specific string, but for any string with some features
- Regular expressions are a construct to do this

Finding a string

- You already know several ways to find a substring inside a larger string
- `site in sequence` -- tests if the substring `site` is found anywhere in `sequence`
- `sequence.find(site)` -- finds the index of the first occurrence of `site` in `sequence`, returns -1 if not found
- `sequence.count(site)` -- counts the number of times `site` occurs in `sequence` (no overlaps)

Is it a p53 sequence?

- p53 is a tumor antigen
-
```
>>> p53 = "MCNSSCMGGMNRR"
>>> protein =
"SEFTTVLYNFMCNSSCMGGMNRRPILTIIS"
>>> protein.find(p53)
10
>>> protein[10:10+len(p53)]
'MCNSSCMGGMNRR'
>>>
```

p53 needs more than one test

- p53s are variable in one residue
- MCNSSCMGGMNRR
  or
  MCNSSCVGGMNRR
- We can test for both cases, but…
- ... if there are more possibilities then the number of patterns may get really large
- Writing all of them out may become tedious and error prone

Using patterns

- Rather than writing each alternative, we can write a *pattern* capturing at once all the strings we are looking for
- MCNSSCMGGMNRR
  or
  MCNSSCVGGMNRR

  ⇒ MCNSSC[MV]GGMNRR
- We use [] to denote a list of residues: any of them can make the match succeed
- E.g., [FILAPVM] matches any hydrophobic residue

Antennapedia

- 'Homeobox' antennapedia-type protein signature is a string with the pattern below
- [LIVMFE][FY]PWM[KRQTA]
- Starts with any of L, I, V, M, F, E
- Then either F or Y
- Then exactly PWM
- Then any of K, R, Q, T, A

Finding Antennapedia

- Could you find a string matching
  [LIVMFE][FY]PWM[KRQTA]
  in the set below?
- MDPDCFAMSS YQFVNSLASC YPQQMNPQQN HPGAGNSSAG GSGGGAGGSG GVVPSGGTNG
  GQGSAGAATP GANDYFPAAA AYTPNLYPNT PQPTTPIRRL ADREIRIWWT TRSCSRSDCS
  CSSSSNSNSS NMPMQRQSCC QQQQQLAQQQ HPQQQQQQQQ ANISCKYAND PVTPGGSGGG
  GVSGSNNNNN SANSNNNNSQ SLASPQDLST RDISPKLSPS SVVESVARSL NKGVLGGSLA
  AAAAAAGLNN NHSGSGVSGG PGNVNVPMHS PGGGDSDSES DSGNEAGSSQ NSGNGKKNPP
  QIYPWMKRVH LGTSTVNANG ETKRQRTSYT RYQTLELEKE FHFNRYLTRR RRIEIAHALC
  LTERQIKIWF QNRRMKWKKE HKMASMNIVP YHMGPYGHPY HQFDIHPSQF AHLSA
- Probably not easily
- Computers can help here

Antennapedia sequences

- Here are some sequences which contain substrings which fit the pattern [LIVMFE][FY]PWM[KRQTA]
- ...LHNEANLRIYPWMRSAGADR...
- ...PTVGKQIFPWMKES...
- ...VFPWMKMGGAKGGESKRTR...

Regular expressions

- Regular expressions (also called regexps) are a format of patterns to describe sets of strings
- Regular comes from the fact that they can describe infinite sets regular enough to have a finite description
- Expressions comes from the fact that they can be built using operators like mathematical expressions

Many different languages for regexps

- Even if they are all based on the same theoretical core, there are a number of slightly different languages for regexps
  - Grep, emacs, awk, POSIX, and the shells all use slightly different notations
  - Some biological databases also define their own format
- The most common language for regexp is the perl5 regular expression language
- It is used by python and other languages as well
- We fill focus on it

Regexps in python

- Regexps can be used in python via the re library
-
```
>>> from re import *
>>> text = "My name is Andrew"
>>> result = search(r"[AT]",text)
>>> print(result)
<_sre.SRE_Match object; span=(11, 12), match='A'>
```
- The `search` function takes a pattern and a string, and returns a (not easily readable) match object
- The `r` in `r"[AT]"` denotes it is a raw string, that is escape sequences are dealt with as normal characters
  - Not useful here, but it is a good practice since many patterns use `\`
- We looked for either a `A` or a `T`

Using a match object

```
>>> from re import *
>>> text = "My name is Andrew"
>>> result = search(r"[AT]",text)
>>> print(result.start())
11
>>> print(result.end())
12
>>> print(text[result.start():result.end()])
A
```

- You can find the start and end of the first match, and then extract the matching substring

Match a protein motif

```
>>> pattern = r"[LIVMFE][FY]PWM[KRQTA]"
>>> seq = "LHNEANLRIYPWMRSAGADR"
>>> match = search(pattern, seq)
>>> print(match.start())
8
>>> print(match.end())
14
>>> print(seq[match.start():match.end()])
IYPWMR
```

And when there is no match?

- The search function will return `None` if no match is found
-
```
>>> pattern = r"[LIVMFE][FY]PWM[KRQTA]"
>>> match = search(pattern,"AAAAAAAAAAA")
>>> print(match)
None
```

List matching patterns

```
from re import *
pattern = r"[LIVMFE][FY]PWM[KRQTA]"
sequences = ["LHNEANLRIYPWMRSAGADR",
             "PTVGKQIFPWMKES",
             "NEANLKQIFPGAATR",
             "VFPWMKMGGAKGGESKRTR"]
for seq in sequences:
    match = search(pattern, seq)
    if match:
        print (seq, "matches")
    else:
        print (seq, "does not have the motif")
```

`Match` can be used as a Boolean

- Sample execution:

```
LHNEANLRIYPWMRSAGADR matches
PTVGKQIFPWMKES matches
NEANLKQIFPGAATR does not have the motif
VFPWMKMGGAKGGESKRTR matches
```

Negation

- Suppose you know from structural reasons that a residue cannot be a proline. You could write

  `[ACDEFGHIKLMNQRSTVWY]`

- That's tedious, there is a shorter option: `[^P]`
- `[^P]` matches anything which is not a proline

N-glycosylation site

- This is the pattern for PS00001, ASN_GLYCOSYLATION
- N[^P][ST][^P]
- Match an N,
  Then anything which isn’t a P,
  Then an S or T,
  And finally, anything which isn’t a P

Allowing anything

- Sometimes we can allow anything on a given position, just the number of elements is relevant
- We can of course use `[ACDEFGHJKLMNPQRSTVWY]`
- This is quite tedious, python allows one to use just `.`
- Thus, `P.P` matches a proline followed by any residue followed by another proline
- If you want to really require a . then you need to prefix it with a \, and the same for other special characters

Barwin domain signature 1

- The pattern is: `CG[KR]CL.V.N`

- The substring must start with a C,
  second letter must be a G,
  third must be a K or R,
  fourth must be a C,
  fifth must be an L,
  sixth may be any residue,
  seventh must be a V,
  eight may also be any residue,
  last must be an N.

...SSCGKCLSVTNTG...

Repetitions

- Sometimes a pattern occurs multiple times
- For example, a pattern may require 5 hydrophobic residues in a row
- You could write it as

  `[FILAPVM][FILAPVM][FILAPVM][FILAPVM][FILAPVM]`
- This is tedious again, python provides a shorter notation
- Writing {n} after the pattern requires to repeat it n times
- Thus the pattern above becomes `[FILAPVM]{5}`

`[FILAPVM]{5}`

- The {} repeats the previous pattern
- The pattern `[FILAPVM]{5}` find matches in all of the following
- AAAAA
  AAPAP
  LAPMAVAILA
  VILLAMAP
  LAPLAMP
- And `.{6}` find matches in any string of at least length 6

EGF-like domain signature 1

- The pattern for PS00022 is: `C.C.{5}G.{2}C`
- Match a C, followed by any residue, followed by a C, followed by 5 residues of any type, then a G, then 2 residues of any type, then a C
- ...VCSNEGKCICQPDWTGKDCS...

Grouping

- If you want to specify longer patterns to repeat, you can use parenthesis
- The pattern `(PP[VM]){2}` find matches in all of the following
- PPVPPV
  PPVPPM
  PPMPPV
  PPMPPM
- We will come back to groups later on

Count ranges

- Sometimes you may want a range of repetitions
- For instance, a loop may have 3 to 5 residues in it
- All the patterns seen so far only matched a fixed number of characters, so we need to extend the notation
- {m,n} - repeat the previous pattern at least m times and up to n times
- For example, `A{3,5}` matches AAA, AAAA, and AAAAA but does not match AA nor AATAA.

EGF-like domain signature 2

- The pattern for PS01186 is:
  `C.C.{2}[GP][FYW].{4,8}C`
- Use a spacer of at least 4 and up to (and including) 8 residues
- RHCYCEEGWAPPDCTTQLKA
  RHCYCEEGWAPPDECTTQLKA
  RHCYCEEGWAPPDEQCTTQLKA
  RHCYCEEGWAPPDEQWCTTQLKA
  RHCYCEEGWAPPDEQWICTTQLKA

Special count ranges

- A few common count ranges have a special notation
- `?` means 0 or 1 (equivalent to `{0,1}`)
- `*` means 0 or more
- `+` means 1 or more
- These are very common also outside bioinformatics and regexps (e.g., the `*` for an unbounded number of parameters in function definitions)

Beginning and end of sequences

- We may want to look for patterns only at the beginning or at the end of a sequence
- There is a notation for this as well
- `^` (not inside a `[]`) means the start of the sequence
  - `^` inside a `[]` means “not”
- `$` means the end of the sequence

Neuromodulin (GAP-43) signature 1

- The pattern for PS00412 is: `^MLCC[LIVM]RR`
- It does match: MLCCIRRTKPVEKNEEADQE
- It does not match: MMLCCIRRTKPVEKNEEADQE

Endoplasmic reticulum targeting sequence

- The pattern for PS00014 is: `[KRHQSA][DENQ]EL$`
- It does match: ADGGVDDD`HDEL`
- It does not match: ADGGVDDDHDELQ

Groups, again

- Suppose an enzyme modifies a protein, and recognizes the portion of the sequence matching `[ASD]{3,5}[IL][^P]{2,5}`
- The modification only occurs on the `[IL]` residue.
- Thus we want to know the position of that one residue inside the matching
- We can use groups to this end

Finding the residue

- ```
  >>> from re import *
  >>> pattern = r"[ASD]{3,5}([LI])[^P]{2,5}"
  >>> seq = "EASALWTRD"
  >>> match = search(pattern, seq)
  >>> print(match)
  <_sre.SRE_Match object; span=(1, 9),
  match='ASALWTRD'>
  >>> print(match.start(),match.end())
  1 9
  >>> print(match.start(1),match.end(1))
  4 5
  ```
- We can use the parameter of `start` and `end` to select a specific group
  ```

Parsing with regexps

- Groups are great for parsing
- Suppose I have the string

  `Name: Andrew Age: 33`

- We want to extract from the string the name and the age values
- We can use a pattern with a group for each field
- `"Name: ([^ ]+) +Age: ([0123456789]+)"`

Dissecting the pattern

- `"Name: ([^ ]+) +Age: ([0123456789]+)"`
- Starts with `Name:`
  Then any sequence of at least one non space (group 1)
  Then at least one space
  Then `Age:`
  Then any sequence of digits (group 2)

Some more shorthands

- There are shorthands for the most used sets
- `\d` = `[0123456789]`
- `\w` = letters, digits, and the underscore
- `\s` = “whitespace” (space, newline, tab, and a few others)
- For these it is important to use raw strings

Using the pattern

- ```
  from re import *
  text = "Name: Andrew Age: 33"
  pattern = r"Name: ([^ ]+) +Age: (\d+)"
  match = search(pattern, text)
  print(match.start(1),match.end(1))
  print(match.group(1),match.group(2))
  ```
- Sample execution:
  ```
  6 12
  Andrew 33
  ```
  ```

Exercises

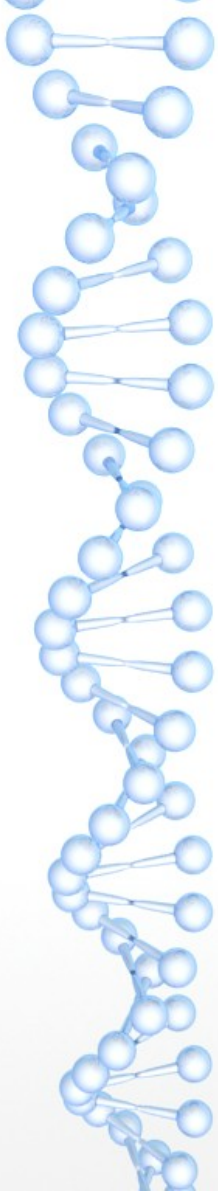
- Define a pattern matching only the 4 strings below
  AAAATGC
  AAATCC
  AAAATCC
  AAATGC
- Look for the pattern in the string TCAAATGCTT

Exercises

- What does the pattern "`[GT]C{4}T`" means?
- Count how many strings in the file sequences.seq match it

Exercise

- Use regexps and groups to parse the file weights.txt to create the corresponding dictionary



Inheritance

Ivan Lanese

Inheritance

- You may define classes starting from other classes
- E.g., a student is a person (and as such has a name, an address, …) but has some more information (e.g., list of exams) and behaviors (e.g., pass an exam)
- You can define student as subclass of person, person is called a superclass of student
- This allows you to reuse part of your code

Subclasses

- Subclasses have their own constructor
- Subclasses inherit instance variables and methods from the superclass
- Subclasses may have additional instance variables
- Subclasses may define methods:
  - If they have the same name as a method from the superclass, the latter is replaced
  - If they have a new name, the method is added

Subclasses: syntax

- You only need to put in parenthesis the name of the superclass, then it is a normal class definition
- `class student(person):`
- You can also access overloaded methods from the superclass
- `def __str__(self):`

```
        return(super().__str__()+" other stuff specific of the subclass")
```

Subclasses vs objects as instance variables

- You use subclasses when objects are objects of the superclass as well
- A student is a person, hence it makes sense to have student as subclass of person
- You use objects as instance variables in all the other cases
- An exam is not a student, hence exam should not be a subclass of students
- But it makes sense for a student to have an instance variable exam (or better list of exams)

Class hierarchies

- Subclasses can be superclasses of other classes
- Student can be a subclass of person, and thesis student can be a subclass of student
- You can build complex hierarchies
- In python, every class actually inherits from `object`
- E.g., it will have (not very useful) methods `__eq__` and `__str__`

Exercise

- Define a class `bankAccount` with an amount and methods to deposit and withdraw money, and to show the status. Define a subclass `interestsAccount` with an additional method which increases the amount by 2%. The interest rate should also be part of the status.

Exercise

- Define a class `Person` with a constructor, and methods `birthday` and `__str__`.
- Define a subclass `Student` of `Person` which also has a list of passed exams (a class themselves, including score and number cfu) and a method to pass the exam. The method `__str__` should also provide information on passed exams.
- Create a student, let him pass two exams and have a birthday. At each step print its information.

Last comment

- We have seen the basics of programming, of python, and of object-orientation
- There is much more
  - You will see further aspects in next module and other courses
  - You will need to discover many things from yourself
- But hopefully you now have some bases to build on
- Good luck!